



Universidade Federal do Rio Grande do Norte
Departamento de Informática e Matemática Aplicada
DIM0124 - Programação Concorrente

Relatório de Desenvolvimento de Aplicação Concorrente

Algoritmo K-Means

Matrícula: 20210081823

Nome: Rary Emanuel Gonçalves Coringa

Contents

1	Introdução	3
1.1	Descrição do Algoritmo	3
1.2	Descrição da Linguagem Erlang	3
2	Configuração dos Experimentos	4
2.1	Configurações do Computador	4
2.1.1	Hardware	4
2.1.2	Software	4
2.2	Configurações de Testes	4
2.2.1	Microbenchmark (JMH)	4
2.2.2	Macrobenchmark (JMeter)	5
2.2.3	Stress Test (JCStress)	6
2.2.4	Profile (JFR)	7
2.2.5	Microbenchmark em Erlang	7
2.2.6	Profile em Erlang	7
3	Implementação Serial	8
3.1	Implementação em Java	8
3.1.1	Descrição	8
3.1.2	Resultados	10
3.2	Implementação em Erlang	12
3.3	Discussão	12
4	Implementação Concorrente	13
4.1	Implementação em Java	13
4.1.1	Descrição	13
4.1.2	Resultados	15
4.2	Discussão	18
5	Implementação Concorrente — Dados em Memória	20
5.1	Implementação em Java	20
5.1.1	Descrição	20
5.1.2	Resultados	20
5.2	Implementação em Erlang	22
5.2.1	Descrição	22
5.2.2	Resultados	22
5.3	Discussão	23
6	Concorrência Estruturada	25
6.1	Implementação em Java	25
6.1.1	Descrição	25
6.1.2	Resultados	25
6.2	Discussão	28

7	Fork/Join — Divisão e Conquista	29
7.1	Implementação em Java	29
7.1.1	Descrição	29
7.1.2	Resultados	29
7.2	Discussão	31
8	Spark — Processamento Distribuído	33
8.1	Implementação em Java	33
8.1.1	Descrição	33
8.1.2	Resultados	35
8.2	Discussão	37
9	Discussão Final	39
9.1	Comparativo Geral das Abordagens	39
9.2	Pontos Positivos e Negativos de Cada Abordagem	42
9.3	Lições Aprendidas	44
10	Repositórios	46

1 Introdução

1.1 Descrição do Algoritmo

O K-Means é um algoritmo de agrupamento não supervisionado que particiona um conjunto de dados em k grupos (clusters), de forma que cada ponto pertença ao cluster cujo centróide é o mais próximo. O algoritmo opera em iterações: na etapa de **atribuição**, cada ponto é associado ao centróide mais próximo segundo a distância euclidiana; na etapa de **atualização**, cada centróide é recalculado como a média aritmética de todos os pontos a ele atribuídos. Esse ciclo se repete até que os centróides estabilizem (deslocamento máximo abaixo de uma tolerância) ou o número máximo de iterações seja atingido.

A complexidade por iteração é $O(N \cdot k \cdot d)$, onde N é o número de pontos, k o número de clusters e d a dimensionalidade dos dados. Por percorrer todos os pontos a cada iteração, o algoritmo é naturalmente paralelizável na etapa de atribuição, tornando-o um caso de estudo relevante para comparação entre estratégias de concorrência.

1.2 Descrição da Linguagem Erlang

Erlang é uma linguagem funcional criada pela Ericsson, projetada para sistemas distribuídos e tolerantes a falhas. Executa sobre a máquina virtual BEAM, que provê concorrência nativa por meio de processos leves e isolados.

Modelo de programação. Utiliza passagem de mensagens (modelo de atores). Processos não compartilham memória e se comunicam exclusivamente via caixas postais, eliminando por construção os problemas de acesso concorrente a estado compartilhado.

Modelo de memória e visibilidade. Cada processo possui heap privado. Mensagens são copiadas ao ser enviadas, garantindo isolamento total. Por não existir memória compartilhada mutável, problemas de visibilidade comuns em Java — como o uso de `volatile` — não se aplicam.

Controle de região crítica. Não existem mutexes ou semáforos. O acesso exclusivo a recursos é obtido encapsulando-os em um processo dedicado que atende requisições uma a uma.

Coleta de lixo. O GC opera por processo, de forma independente, sem pausas globais que afetam toda a aplicação. Processos de vida curta têm seus heaps simplesmente descartados ao terminar.

JIT. A partir do OTP 24, a BEAM inclui um compilador JIT que gera código nativo no carregamento dos módulos, com ganhos típicos de 20–40% em relação ao modo interpretado.

2 Configuração dos Experimentos

2.1 Configurações do Computador

Esta seção descreve o ambiente de hardware e software utilizado na execução dos experimentos. O conhecimento dessas configurações é essencial para a reprodutibilidade dos resultados e para a correta interpretação das métricas coletadas.

2.1.1 Hardware

CPU: O processador utilizado é o **Intel Core i5-13450HX** (13ª geração), com arquitetura híbrida composta por 6 P-cores (Performance, com Hyper-Threading) e 4 E-cores (Efficiency, sem Hyper-Threading), totalizando 10 núcleos físicos e 16 threads lógicas. A frequência máxima é de 4,6 GHz. As memórias cache são: L1d 416 KiB, L1i 448 KiB, L2 9,5 MiB e L3 20 MiB.

RAM: A memória RAM total é de **16 GB DDR5** em configuração dual-channel (2 × 8 GB).

Disco: O armazenamento em disco utilizado é um **SSD NVMe** com capacidade de 512,1 GB.

2.1.2 Software

Sistema Operacional: **Ubuntu 24.04.4 LTS**, kernel 6.17.0-22-generic.

Java: **OpenJDK 25.0.1** - Temurin-25.0.1+8 (build 25.0.1+8-LTS).

Erlang: **Erlang/OTP 25**, emulador BEAM versão 13.2.2.5, com JIT AsmJit habilitado.

2.2 Configurações de Testes

2.2.1 Microbenchmark (JMH)

Os microbenchmarks foram implementados com **JMH 1.37** sobre **JDK 25.0.1**, com modo **AverageTime**, 1 iteração de aquecimento de 10 s, 3 iterações de medição de 10 s e 1 fork por benchmark. O dataset é carregado uma única vez antes das medições e uma semente fixa garante reprodutibilidade dos pontos de teste.

ID	Método	Objetivo
B1	b1_fullRun	Execução completa ($k=10$, maxIter=20, tol= 10^{-6}) — número primário do relatório
B2	b2_perIterationCost	Execução curta ($k=10$, maxIter=5, tol=0) — isola custo por iteração
B3	b3_closestCentroid	Chamada isolada de closestCentroid — ponto quente do laço interno
B4	b4_squaredDistance	Chamada isolada de squaredDistance — baseline aritmético

Table 1: Casos de benchmark JMH definidos em KMeansBenchmark.

A Figura 1 apresenta a classe de benchmark com as anotações de configuração e os quatro métodos medidos.

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.SECONDS)
@Warmup(iterations = 1, time = 10)
@Measurement(iterations = 3, time = 10)
@Fork(1)
public class KMeansBenchmark {

    @Benchmark
    public KMeansResult b1_fullRun(DatasetState s) throws IOException {
        return KMeans.cluster(s.path, s.dataset, 10, 20, 1e-6);
    }

    @Benchmark
    public KMeansResult b2_perIterationCost(DatasetState s) throws IOException {
        return KMeans.cluster(s.path, s.dataset, 10, 5, 0);
    }

    @Benchmark
    public int b3_closestCentroid(HotspotState s) {
        return KMeans.closestCentroid(s.point, s.centroids);
    }

    @Benchmark
    public double b4_squaredDistance(HotspotState s) {
        return KMeans.squaredDistance(s.left, s.right);
    }
}
```

Figure 1: Configuração JMH e casos de benchmark em KMeansBenchmark.

2.2.2 Macrobenchmark (JMeter)

Os macrobenchmarks foram implementados com **Apache JMeter 5.6.3** utilizando o mecanismo de *Java Sampler* (`AbstractJavaSamplerClient`). A classe `KMeansSampler` executa `KMeans.cluster()` diretamente na JVM do JMeter — sem servidor HTTP intermediário — permitindo que o coletor de GC configurado na inicialização do JMeter se aplique integralmente ao algoritmo.

O GC é selecionado pela variável de ambiente `GC_ALGO` passada ao script de inicialização do JMeter. Foram avaliados três coletores:

- `-XX:+UseG1GC` — G1GC (coletor padrão do JDK 25);
- `-XX:+UseZGC` — ZGC (coletor de baixa pausa);
- `-XX:+UseParallelGC` — ParallelGC (coletor paralelo de alta vazão).

A matriz de testes combina os três coletores com cinco contagens de threads, totalizando 15 execuções independentes:

Parâmetro	Valor
Coletores de GC	G1GC, ZGC, ParallelGC
Threads JMeter	1, 2, 4, 8, 16
Duração por execução	120 s
Ramp-up	30 s
k	10
maxIter	3
tol	0,0
Métrica primária	Throughput (execuções/min)

Table 2: Configuração do macrobenchmark JMeter.

2.2.3 Stress Test (JCStress)

Os testes de condição de corrida foram implementados com **JCStress 0.16** e executados sobre **JDK 25.0.1**. O JCStress executa múltiplos processos paralelos variando configurações de compilação e afinidade de CPU, maximizando a cobertura de possíveis intercalações entre threads.

Parâmetro	Valor
Ferramenta	JCStress 0.16
Preset	<code>default</code>
Forks por teste	6 (1 normal + 5 stress)
Iterações por fork	5
Tempo por iteração	1.000 ms
CPUs utilizadas	16

Table 3: Configuração do stress test JCStress.

Foram definidos quatro testes, cada um associado a uma região crítica do algoritmo:

ID	Classe	Região crítica
JCS1	<code>JCS1_ClosestCentroidTest</code>	Leitura concorrente do snapshot de centróides
JCS2	<code>JCS2_SquaredDistanceTest</code>	Leitura concorrente de <code>double[]</code> compartilhado
JCS3	<code>JCS3_LocalCountAccumulationTest</code>	<code>counts[0]++</code> em array compartilhado (abordagem ingênua)
JCS4	<code>JCS4_LocalSumAccumulationTest</code>	<code>sums[0] += 1.0</code> em array compartilhado (abordagem ingênua)

Table 4: Testes JCStress e regiões críticas associadas.

2.2.4 Profile (JFR)

O profile foi capturado com **JFR (Java Flight Recorder)**, ferramenta embutida no JDK 25 — sem necessidade de instalação adicional. A gravação foi iniciada via flag de JVM na execução do algoritmo e analisada exclusivamente com a ferramenta de linha de comando `jfr` do JDK, sem uso do JMC (*Java Mission Control*).

A configuração `settings=profile` habilita amostragem de CPU em alta resolução (intervalo de 10 ms), adequada para identificar hot methods. A flag `dumponexit=true` garante que o arquivo `.jfr` seja gravado ao final da execução mesmo sem intervenção manual.

Parâmetro	Valor
Ferramenta	JFR (JDK 25 built-in)
Configuração	<code>settings=profile</code>
Captura	<code>dumponexit=true</code>
Análise	<code>jfr print</code> (CLI)
GC	G1GC
k	10
<code>maxIter</code>	3
Dataset	≈ 1 GB

Table 5: Configuração do profile JFR.

Os eventos analisados cobrem quatro áreas:

- amostras de CPU por método (identificação dos hot methods);
- pausas e uso de heap do coletor de GC;
- carga de CPU por thread e número de threads ativas;
- eventos de leitura de arquivo (bytes lidos, duração).

2.2.5 Microbenchmark em Erlang

Os microbenchmarks Erlang utilizam `timer:tc/1`, função nativa do OTP que mede o tempo de execução de uma fun com resolução de microssegundos. São executadas 1 iteração de aquecimento e 3 de medição, com o dataset carregado uma única vez em memória antes de qualquer medição para eliminar o custo de I/O dos resultados. São medidos três casos equivalentes ao JMH: execução completa do algoritmo (B1), `closest_centroid` isolado (B3) e `squared_distance` isolado (B4).

2.2.6 Profile em Erlang

O profile Erlang coleta métricas de tempo e GC via funções nativas do OTP, sem necessidade de ferramentas externas. São capturados: tempo de CPU, tempo de parede, número de coletas GC e utilização média dos schedulers BEAM. O profile cobre uma execução completa do algoritmo com $k = 10$ e `maxIter=10`.

3 Implementação Serial

3.1 Implementação em Java

3.1.1 Descrição

A implementação serial em Java é composta por quatro classes principais, cada uma com responsabilidade bem delimitada:

- **CsvDataset** — lê o cabeçalho do arquivo CSV e expõe os dados linha a linha via `forEachPoint`, sem carregar o dataset inteiro em memória;
- **KMeans** — contém o algoritmo K-Means: inicialização dos centróides, laço de atribuição e recomputo, verificação de convergência e avaliação final;
- **KMeansResult** — registro imutável que armazena os centróides finais, tamanhos de cluster, número de iterações e soma dos erros quadráticos;
- **Main** — ponto de entrada; configura $k = 10$, `maxIter = 10` e `tolerance = 10^{-6}` e imprime o resumo dos clusters ao final.

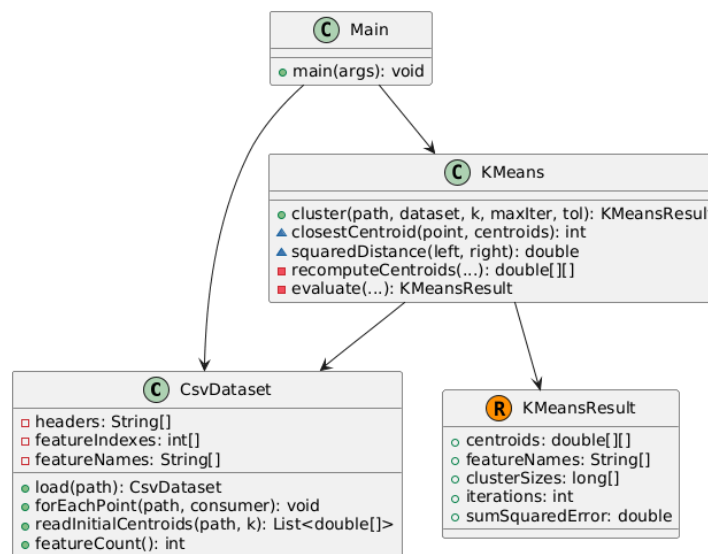


Figure 2: Diagrama de classes da implementação serial em Java.

O dataset é processado em modo *streaming*: a cada iteração do algoritmo, `forEachPoint` abre o arquivo, lê e descarta o cabeçalho, e entrega cada linha decodificada ao consumidor sem acumular pontos na memória heap. A Figura 3 mostra essa interface.

```

public void forEachPoint(Path path, PointConsumer consumer) throws IOException {
    try (BufferedReader reader = Files.newBufferedReader(path)) {
        reader.readLine();

        String line;
        long rowNum = 1;

        while ((line = reader.readLine()) != null) {
            rowNum++;

            if (line.isBlank()) {
                continue;
            }

            consumer.accept(parse(line), rowNum);
        }
    }
}

```

Figure 3: Leitura linha a linha do dataset (`CsvDataset.forEachPoint`).

O laço principal do algoritmo, mostrado na Figura 4, percorre todos os pontos a cada iteração para acumular somas parciais e contagens por cluster, e em seguida recomputa os centróides dividindo as somas pelas contagens.

```

double[][] sums = new double[k][dataset.featureCount()];
long[] counts = new long[k];
double[][] activeCentroids = centroids;

dataset.forEachPoint(datasetPath, (point, rowNum) -> {
    int closest = closestCentroid(point, activeCentroids);
    counts[closest]++;
    addPoint(sums[closest], point);
});

double[][] updatedCentroids = recomputeCentroids(centroids, sums, counts);

```

Figure 4: Laço de atribuição e recomputo de centróides (`KMeans.cluster`).

A função `closestCentroid` (Figura 5) varre os k centróides para cada ponto e retorna o índice do mais próximo usando distância euclidiana ao quadrado, evitando a raiz quadrada desnecessária. Essa função é o principal ponto quente da CPU, executada $N \times k$ vezes por iteração.

```

static int closestCentroid(double[] point, double[][] centroids) {
    int bestIndex = 0;
    double bestDistance = squaredDistance(point, centroids[0]);

    for (int i = 1; i < centroids.length; i++) {
        double distance = squaredDistance(point, centroids[i]);

        if (distance < bestDistance) {
            bestDistance = distance;
            bestIndex = i;
        }
    }

    return bestIndex;
}

static double squaredDistance(double[] left, double[] right) {
    double total = 0.0d;

    for (int i = 0; i < left.length; i++) {
        double delta = left[i] - right[i];
        total += delta * delta;
    }

    return total;
}

```

Figure 5: Busca do centróide mais próximo e distância euclidiana ao quadrado.

3.1.2 Resultados

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1 (OpenJDK 64-Bit Server VM), com 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada. O dataset utilizado possui aproximadamente 1 GB de dados de pedidos sintéticos.

ID	Benchmark	Média	Erro ($\pm$)	Unidade
B1	Execução completa ($k=10$, $\text{maxIter}=20$)	212,57	34,48	s/op
B2	Custo por iteração ($k=10$, $\text{maxIter}=5$)	60,05	2,46	s/op
B3	<code>closestCentroid</code> ($k=10$, 38 dims)	96,82	5,59	ns/op
B4	<code>squaredDistance</code> (38 dims)	9,31	1,41	ns/op

Table 6: Resultados JMH para a implementação serial em Java.

B2 mostra que cada iteração leva cerca de 12 s, dominado pela releitura do CSV a cada ciclo. B3 e B4 mostram que o cálculo aritmético é rápido (96 ns e 9 ns por chamada). O gargalo não está na CPU, mas na leitura e no parsing do arquivo.

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 sobre JDK 25.0.1, com $k = 10$, $\text{maxIter} = 3$ e $\text{tol} = 0,0$. Cada configuração (GC $\times$ threads) rodou por 120 s com ramp-up de 30 s. A Tabela 7 apresenta a throughput obtida em cada combinação.

Threads	G1GC	ZGC	ParallelGC	Unidade
1	1,56	1,50	1,52	execuções/min
2	2,25	2,01	2,26	execuções/min
4	2,70	2,54	2,79	execuções/min
8	3,44	3,23	3,51	execuções/min
16	4,66	4,40	4,85	execuções/min

Table 7: Throughput por número de threads e coletor de GC — implementação serial em Java.

O throughput cresce com o número de threads JMeter, mas com retornos decrescentes: de 1 para 16 threads, o ganho é de $\approx 3\times$. A latência piora com mais threads, pois todas as instâncias leem o mesmo arquivo de ≈ 1 GB ao mesmo tempo, saturando a I/O do disco.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo serial ($k = 10$, `maxIter=3`, G1GC, dataset ≈ 1 GB). A gravação durou 118 s e foi analisada com a ferramenta `jfr` do JDK 25.

Hot Methods. A Tabela 8 apresenta os métodos com maior frequência de amostragem de CPU (`jdk.ExecutionSample`, 11.054 amostras no total).

Método	Amostras	CPU (%)	Categoria
<code>FloatingDecimal.readJavaFormatString</code>	2784	25,2	Parsing CSV
<code>String.split</code>	1873	16,9	Parsing CSV
<code>FloatingDecimal.copyDigits</code>	1817	16,4	Parsing CSV
<code>FloatingDecimal.parseDouble</code>	1782	16,1	Parsing CSV
<code>FloatingDecimal.doubleValue</code>	831	7,5	Parsing CSV
<code>KMeans.squaredDistance</code>	638	5,8	Algoritmo
<code>BufferedReader.readLine</code>	609	5,5	I/O
<code>KMeans.closestCentroid</code>	205	1,9	Algoritmo
<code>KMeans.addPoint</code>	33	0,3	Algoritmo

Table 8: Hot methods na implementação serial em Java (JFR, 11.054 amostras).

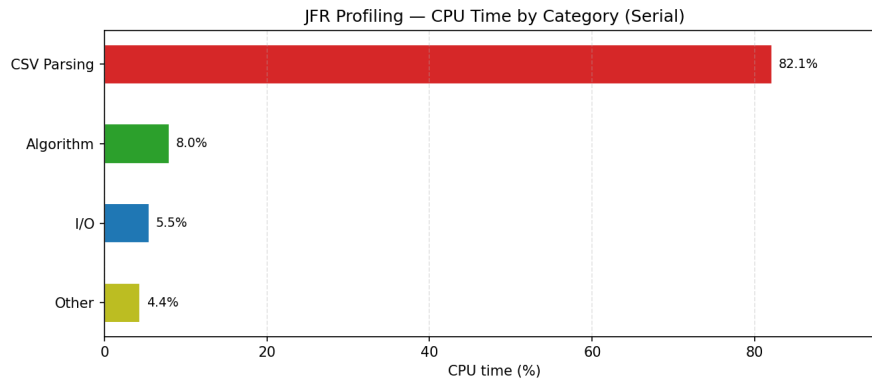


Figure 6: Distribuição do tempo de CPU na execução serial (JFR, 11.054 amostras): o parsing CSV concentra $\approx 82\%$ do tempo total, deixando apenas $\approx 8\%$ para o algoritmo K-Means.

O parsing CSV (`String.split` + `Double.parseDouble` e suas dependências internas) consome $\approx 82\%$ do tempo de CPU, revelando que o gargalo real não é a leitura de disco em si, mas a conversão de texto para `double` linha a linha. O código aritmético do K-Means (`squaredDistance` + `closestCentroid` + `addPoint`) representa apenas $\approx 8\%$ — coerente com os resultados JMH de B3 (96 ns/op) e B4 (9 ns/op).

GC. Foram realizadas 1.099 coletas de GC em 118 s, com pausa total de 805 ms — apenas 0,68% do tempo de execução. O heap oscila entre ≈ 150 MB e ≈ 5 MB após cada coleta, reflexo da alta taxa de criação e descarte de objetos `String` durante o parsing. O GC não é um gargalo.

Threads. Uma única thread de trabalho executa todo o algoritmo; as demais 7 threads são internas da JVM (GC, compilador JIT, etc.). Não há contenção nem paralelismo.

Síntese. O principal gargalo é o parsing CSV (`String.split` + `Double.parseDouble`), que consome $\approx 82\%$ do CPU. O cálculo aritmético do K-Means representa apenas $\approx 8\%$.

3.2 Implementação em Erlang

Uma implementação serial em Erlang não foi realizada. A linguagem é projetada em torno do modelo de atores: processos são a unidade fundamental de execução e toda computação significativa ocorre por troca de mensagens entre eles. Escrever um K-Means verdadeiramente serial em Erlang exigiria contornar deliberadamente os mecanismos centrais da linguagem, produzindo código artificial e não representativo. A implementação Erlang é apresentada diretamente na Seção 4 como versão concorrente.

3.3 Discussão

Trechos caros e candidatos à concorrência. O JFR mostra que $\approx 82\%$ do CPU é gasto com parsing CSV (`String.split` + `Double.parseDouble`). O laço de atribuição, que chama `closestCentroid` $N \times k$ vezes por iteração, é independente entre pontos e é o principal candidato à paralelização. A releitura do arquivo a cada iteração (`forEachPoint`) é outro gargalo que poderia ser eliminado carregando o dataset em memória.

Comparação de coletores de GC. O ParallelGC tem o maior throughput em todos os níveis de carga, seguido pelo G1GC. O ZGC fica levemente atrás em todos os casos pelas barreiras de escrita do seu algoritmo. No geral, o impacto do GC é pequeno — as pausas somam menos de 1% do tempo total.

4 Implementação Concorrente

4.1 Implementação em Java

4.1.1 Descrição

A estrutura geral do algoritmo é mantida em relação à versão serial: inicializar centróides, iterar entre atribuição e recomputo, verificar convergência. O que muda é *como* os pontos são processados dentro de cada iteração.

Na versão serial, a thread principal lê cada ponto diretamente do arquivo, chama `closestCentroid` e acumula os resultados em arrays compartilhados. Na versão concorrente, a leitura ainda é feita sequencialmente, mas os pontos são agrupados em lotes (`BATCH_SIZE = 5.000`) e cada lote é submetido ao pool de workers como uma tarefa independente. Os workers acumulam em arrays locais e retornam um `PartialResult`; a thread coordenadora reduz todos os parciais após coletar os `Futures`.

Esse padrão — dispatch em lotes, acumulação local, redução sequencial — elimina a necessidade de sincronização sem uso de `synchronized` ou variáveis atômicas. A coordenadora aguarda todos os resultados antes de prosseguir, garantindo que não há leitura de dados ainda sendo escritos.

Uma cópia dos centróides é tirada antes do início de cada iteração, garantindo que todos os workers trabalhem com o mesmo estado durante aquela iteração.

Foram implementadas três variantes concorrentes:

- **v2-platform** — pool fixo de N platform threads para os workers e a thread `main` como coordenadora.
- **v2-virtual** — pool fixo de N virtual threads para os workers, mesma estrutura de `v2-platform`.
- **v2-hybrid** — pool fixo de N platform threads como workers e uma virtual thread como coordenadora. A virtual thread suspende automaticamente durante operações de I/O e durante a espera pelos resultados dos workers, liberando o processador para outras tarefas.

As classes `CsvDataset`, `KMeansResult` e `Main` não foram alteradas. As adições ficaram concentradas em `KMeans.java`: dois records privados (`PartialResult`, `EvaluatePartial`) e três métodos novos (`runClustering`, `processAssignBatch`, `processEvaluateBatch`). O método `evaluate` foi refatorado para também usar o pool de workers. A classe `KMeansSampler` foi adicionada como adaptador `JMeter` para os macrobenchmarks.

O diagrama de classes da Figura 7 mostra as classes do projeto com as adições da versão concorrente em destaque.

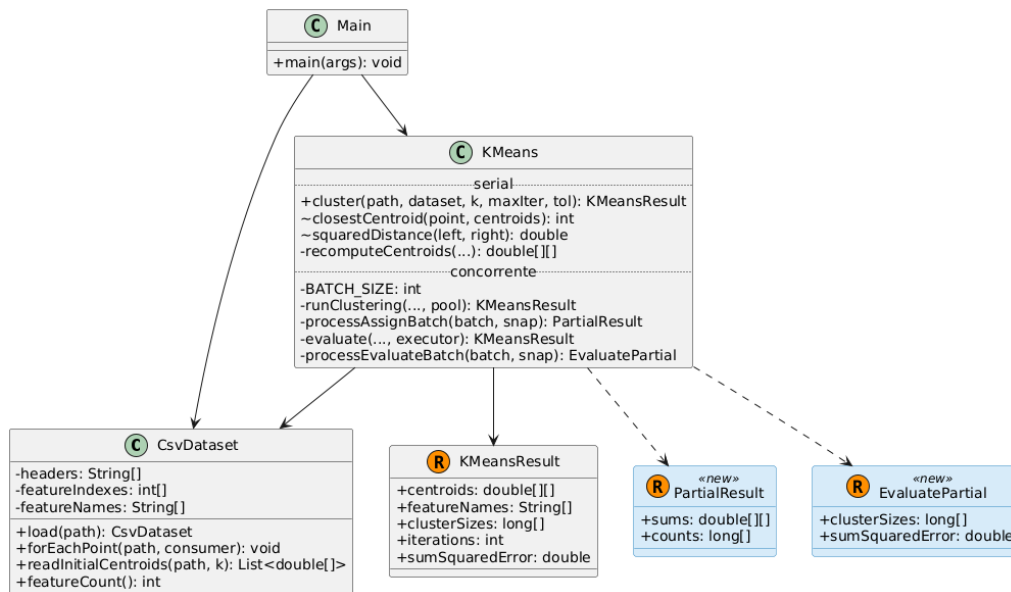


Figure 7: Diagrama de classes da implementação concorrente em Java. Elementos destacados são novos em relação à versão serial.

A Figura 8 mostra a criação dos dois executores e a submissão do método coordenador à virtual thread.

```

int n = Runtime.getRuntime().availableProcessors();
ExecutorService computePool = Executors.newFixedThreadPool(n, Thread.ofPlatform().factory());
ExecutorService coordinator = Executors.newFixedThreadPool(n, Thread.ofVirtual().factory());

try {
    return coordinator.submit(
        () -> runClustering(datasetPath, dataset, k, maxIterations, tolerance,
            centroids, computePool)
    ).get();
} finally {
    coordinator.shutdown();
    computePool.shutdown();
}
  
```

Figure 8: Criação dos executores e submissão do coordenador (`KMeans.cluster`).

A Figura 9 mostra o laço de dispatch de lotes dentro de `runClustering`: a leitura sequencial acumula pontos até atingir `BATCH_SIZE`, quando cada lote é submetido ao pool e o buffer é esvaziado.

```

double[][] snapshot = copy(centroids);
List<Future<PartialResult>> futures = new ArrayList<>();
List<double[]> batch = new ArrayList<>(BATCH_SIZE);

dataset.forEachPoint(datasetPath, (point, rowNum) -> {
    if (batch.size() == BATCH_SIZE) {
        List<double[]> current = List.copyOf(batch);
        futures.add(computePool.submit(() -> processAssignBatch(current,
snapshot)));
        batch.clear();
    }
});

// reduce
for (Future<PartialResult> f : futures) {
    PartialResult pr = f.get();
    for (int c = 0; c < k; c++) {
        counts[c] += pr.counts()[c];
        for (int d = 0; d < sums[c].length; d++)
            sums[c][d] += pr.sums()[c][d];
    }
}

```

Figure 9: Dispatch de lotes e coleta de futures (KMeans.runClustering).

A Figura 10 mostra `processAssignBatch`, o método executado por cada worker: usa arrays locais (`localSums`, `localCounts`) sem compartilhamento, retornando um `PartialResult` imutável.

```

private static PartialResult processAssignBatch(List<double[]> batch,
double[][] centroids) {
    int k = centroids.length, d = centroids[0].length;
    double[][] localSums = new double[k][d];
    long[] localCounts = new long[k];

    for (double[] point : batch) {
        int c = closestCentroid(point, centroids);
        localCounts[c]++;
        addPoint(localSums[c], point);
    }
    return new PartialResult(localSums, localCounts);
}

```

Figure 10: Processamento local de um lote sem arrays compartilhados (KMeans.processAssignBatch).

4.1.2 Resultados

Avaliação de Condição de Corrida Foram identificadas quatro regiões críticas na implementação concorrente, todas relacionadas a acumulações que seriam problemáticas caso múltiplas threads compartilhassem os mesmos arrays de escrita:

ID	Local	Operação	Risco
CR1	processAssignBatch	counts[c]++	lost update em long[]
CR2	processAssignBatch	sums[c][d] += v	lost update em double[] []
CR3	processEvaluateBatch	sse += v	acumulação não atômica de double
CR4	processEvaluateBatch	clusterSizes[c]++	lost update em long[]

Table 9: Regiões críticas identificadas na implementação concorrente.

A Figura 11 mostra `processAssignBatch`, o método executado por cada worker. As operações `localCounts[c]++` (CR1) e `addPoint(localSums[c], point)` — que internamente faz `sums[c][d] += v` (CR2) — seriam race conditions se os arrays fossem compartilhados entre workers. A implementação usa arrays locais por tarefa, tornando-as thread-safe.

```
private static PartialResult processAssignBatch(List<double[]> batch, double[][] centroids) {
    int k = centroids.length, d = centroids[0].length;
    double[][] localSums = new double[k][d]; // local - sem compartilhamento
    long[] localCounts = new long[k]; // local - sem compartilhamento

    for (double[] point : batch) {
        int c = closestCentroid(point, centroids);
        localCounts[c]++; // CR1 - race se fosse array compartilhado
        addPoint(localSums[c], point); // CR2 - race se fosse array compartilhado
    }

    return new PartialResult(localSums, localCounts);
}
```

Figure 11: Operações CR1 e CR2 em `processAssignBatch`: `localCounts[c]++` e `addPoint` seriam race conditions se os arrays fossem compartilhados.

Os testes foram executados com JCSstress 0.16, preset `default`, 6 forks por teste e 5 iterações de 1.000 ms por fork. A Tabela 10 apresenta o resultado consolidado.

ID	Resultado	Outcome FORBIDDEN	Freq. máx.
JCS1	PASSOU	nenhum	—
JCS2	PASSOU	nenhum	—
JCS3	FALHOU (esperado)	"1" lost update em counts[0]++	28,31%
JCS4	FALHOU (esperado)	"1.0" lost update em sums[0]+=1.0	27,10%

Table 10: Resultados JCSstress — regiões críticas da implementação concorrente.

JCS1 e JCS2 não produziram nenhum resultado inválido: a leitura concorrente é segura. JCS3 e JCS4 falharam como esperado: atualizações perdidas ocorrem em até 28% das amostras quando os arrays são compartilhados sem proteção.

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1, com 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada. A Tabela 11 compara as três variantes com a versão serial.

ID (Un.)	Serial	v2-platform	v2-virtual	v2-hybrid
B1 (s)	212,57	214,87 ($\pm 29,10$)	211,64 ($\pm 260,68$)	215,69 ($\pm 59,78$)
B2 (s)	60,05	60,69 ($\pm 6,41$)	71,61 ($\pm 227,05$)	60,44 ($\pm 5,33$)
B3 (ns)	96,82	102,24	101,11	102,07
B4 (ns)	9,31	9,70	9,58	9,87

Table 11: Resultados JMH — serial vs. variantes v2 (média; B1/B2 com erro $\pm$).

Nenhuma variante reduz o tempo em relação ao serial: o parsing CSV sequencial domina B1 e B2 em todas elas. A v2-virtual tem variância muito alta (± 260 s em B1) porque o `ForkJoinPool` não é adequado para tarefas CPU-bound que nunca bloqueiam. B3 e B4 são praticamente inalterados nas três variantes.

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 sobre JDK 25.0.1 ($k = 10$, `maxIter=3`, `tol=0,0`), 120 s por configuração e ramp-up de 30 s. A Tabela 12 mostra o throughput com G1GC para as três variantes.

Threads	v2-platform	v2-virtual	v2-hybrid
1	1,65	1,59	1,60
2	1,95	2,17	2,13
4	3,92	3,29	2,67
8	3,08	4,00	3,44
16	4,49	4,55	4,53

Table 12: Throughput (execuções/min, G1GC) por variante v2 e número de threads JMeter.

Com 1 thread JMeter o throughput é $\approx 1,6$ exec/min para todas as variantes. Com 4 threads, v2-platform lidera (3,92); com 8, v2-virtual supera as demais (4,00) porque suas virtual threads são multiplexadas sobre um número menor de threads reais, evitando a sobrecarga do sistema operacional com muitas threads concorrentes. Em 16 threads todas convergem para $\approx 4,5$ exec/min. O ZGC fica levemente abaixo dos demais em todos os casos pelo custo adicional do seu algoritmo de coleta.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo ($k = 10$, `maxIter=3`, G1GC, dataset ≈ 1 GB). A gravação durou ≈ 113 s e foi analisada com a ferramenta `jfr` do JDK 25.

v2-platform e v2-virtual. Em ambas as variantes, a thread `main` domina o perfil de CPU: 86,6% em v2-platform (parsing CSV) e 99,8% em v2-virtual. Na v2-virtual os workers aparecem em apenas 21 das 10.164 amostras, pois cada lote termina em menos de 10 ms — abaixo do intervalo de amostragem do JFR — deixando as virtual threads invisíveis ao profiler. GC: ≈ 1.000 coletas Young em ≈ 110 s ($< 1\%$ de pausa). Não é gargalo.

Método	CPU (%)	Categoria
KMeans.squaredDistance	83,4 (workers)	Algoritmo
KMeans.closestCentroid	14,2 (workers)	Algoritmo
KMeans.addPoint	1,5 (workers)	Algoritmo
coordenador virtual	0,1	Coordenação

Table 13: Hot methods na implementação v2-hybrid (JFR, 2.133 amostras).

v2-hybrid. O coordenador virtual suspende durante leituras de arquivo e enquanto aguarda os workers, tornando o parsing CSV invisível ao JFR. Os 16 workers de plataforma concentram 99,9% das amostras, com `squaredDistance` e `closestCentroid` no topo. GC: 1.072 coletas em 140 s (0,84% de pausa). Não é gargalo.

4.2 Discussão

Condições de corrida. Os testes JCSstress identificaram quatro regiões críticas (CR1–CR4), todas associadas a acumulações em arrays compartilhados. JCS3 e JCS4 confirmaram lost updates em até 28% das amostras quando a sincronização é omitida. A solução adotada — acumulação local por worker com redução após `Future.get()` — elimina a race condition sem uso de `synchronized` ou `atomics`.

Comparação de abordagens — microbenchmark. A Tabela 14 compara as três variantes com a versão serial.

Abordagem	B1 (s/op)	B2 (s/op)	Erro B1	Erro B2
Serial	212,57	60,05	±34,48	±2,46
v2-platform	214,87	60,69	±29,10	±6,41
v2-virtual	211,64	71,61	±260,68	±227,05
v2-hybrid	215,69	60,44	±59,78	±5,33

Table 14: Comparativo JMH entre abordagens (s/op).

Nenhuma abordagem reduz o tempo médio em relação ao serial: o parsing CSV sequencial é o gargalo comum a todas. v2-virtual apresenta variância muito alta (± 260 s em B1), reflexo do `ForkJoinPool` ao lidar com tarefas CPU-bound. v2-platform e v2-hybrid têm variância próxima ao serial.

Comparação de abordagens — macrobenchmark. A Tabela 15 compara o throughput (G1GC) por número de threads JMeter.

Abordagem	1 t	2 t	4 t	8 t	16 t
v2-platform	1,65	1,95	3,92	3,08	4,49
v2-virtual	1,59	2,17	3,29	4,00	4,55
v2-hybrid	1,60	2,13	2,67	3,44	4,53

Table 15: Throughput (execuções/min, G1GC) por abordagem e número de threads JMeter.

Com poucos threads JMeter (1–2), as três variantes são equivalentes. Com 4 threads, v2-platform tem o maior throughput (3,92 exec/min). Com 8 threads, v2-virtual supera as demais (4,00 exec/min): ao multiplexar workers sobre um número menor de threads

reais, evita a sobrecarga de escalonamento que aparece com muitas threads de plataforma simultâneas. Em 16 threads, todas convergem para $\approx 4,5$ exec/min.

Comparação de coletores de GC. O GC tem impacto pequeno em todas as variantes (pausas $< 1\%$ do tempo total). O ZGC fica levemente abaixo de G1GC e ParallelGC em configurações com muitas threads simultâneas, pelo custo das barreiras de escrita do seu algoritmo. G1GC e ParallelGC têm desempenho semelhante entre si.

5 Implementação Concorrente — Dados em Memória

5.1 Implementação em Java

5.1.1 Descrição

A v3 parte de v2-virtual, a variante com melhor desempenho na seção anterior, e elimina o principal gargalo identificado no profile: o parsing CSV repetido a cada iteração. A mudança central é carregar todos os pontos do dataset em memória uma única vez antes do loop de iterações.

Um novo método carrega o arquivo e retorna todos os pontos como um array em memória. O algoritmo passa a receber esse array diretamente, sem tocar no disco durante o clustering. Nos benchmarks, a leitura do CSV é excluída do tempo medido. No JMeter, todos os threads compartilham uma única cópia dos dados carregada na inicialização.

Em vez de criar lotes dinâmicos, a v3 divide o array em N chunks de tamanho fixo (onde N é o número de processadores lógicos) e usa um `CountDownLatch` para aguardar a conclusão de todos os workers. Cada worker opera sobre sua fatia do array compartilhado (somente leitura) e acumula resultados em arrays locais.

Componente	Alteração
CsvDataset	adicionado <code>loadAllPoints()</code>
KMeans	nova assinatura <code>cluster(double[][], ...)</code> ; <code>CountDownLatch</code> em vez de <code>Futures</code> ; <code>processChunk</code> em vez de <code>processAssignBatch</code> ; <code>DoubleAdder</code> para SSE
KMeansBenchmark	<code>points</code> pré-carregado antes das medições
KMeansSampler	<code>sharedPoints</code> estático compartilhado entre threads JMeter

Table 16: Alterações da v3 em relação à v2-virtual.

5.1.2 Resultados

Avaliação de Condição de Corrida Os mesmos quatro testes JCSstress (JCS1–JCS4) da seção anterior foram executados sobre a v3. A estrutura de acumulação local por worker foi mantida: cada chunk opera em `localSums` e `localCounts` privados, sem compartilhamento.

ID	Resultado	Outcome FORBIDDEN	Freq. máx.
JCS1	PASSOU	nenhum	—
JCS2	PASSOU	nenhum	—
JCS3	FALHOU (esperado)	"1" lost update em <code>counts[0]++</code>	4,05%
JCS4	FALHOU (esperado)	"1.0" lost update em <code>sums[0]+=1.0</code>	5,95%

Table 17: Resultados JCSstress — regiões críticas da v3.

JCS1 e JCS2 passam: leitura concorrente do array de centroides é segura. JCS3 e JCS4 falham como esperado, confirmando que a race condition existe quando os arrays são compartilhados — o que a v3 evita da mesma forma que a v2.

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1 (OpenJDK 64-Bit Server VM), com 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada. O dataset foi pré-carregado em memória antes do início das medições (`@Setup(Level.Trial)`).

ID	Benchmark	Serial	v3	Erro ($\pm$)	Un.
B1	Execução completa ($k=10$, iter=20)	212,57	4,55	17,98	s/op
B2	Por iteração ($k=10$, iter=5)	60,05	1,60	0,17	s/op
B3	closestCentroid (38 dims)	96,82	96,82	—	ns/op
B4	squaredDistance (38 dims)	9,31	9,31	—	ns/op

Table 18: Resultados JMH — serial vs. v3 (dados em memória).

B2 cai de 60 s/op para 1,60 s/op: a eliminação do parsing CSV por iteração reduz o custo em $\approx 37\times$. B1 tem variância elevada (± 18 s) por depender do número de iterações até convergência. B3 e B4 são inalterados (computação pura).

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 sobre JDK 25.0.1, com $k = 10$, $\text{maxIter} = 3$ e $\text{tol} = 0,0$. O dataset é carregado em memória uma única vez por processo JMeter. Cada configuração rodou por 120 s com ramp-up de 30 s, utilizando ParallelGC.

Threads JMeter	Execuções	Throughput (exec/min)	Latência média (ms)
1	105	52,5	1.145
2	112	56,0	2.010
4	116	57,1	3.796
8	119	57,2	7.457
16	134	62,3	13.374

Table 19: Throughput e latência por número de threads JMeter — v3, ParallelGC.

Com 1 thread JMeter, o throughput é 52,5 exec/min. O ganho com mais threads é moderado ($52,5 \rightarrow 62,3$ exec/min de 1 para 16 threads) porque cada execução já satura todos os núcleos internamente. Nenhum erro foi registrado em nenhuma configuração.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo ($k = 10$, $\text{maxIter}=3$, dataset $\approx 1,75$ GB em memória). A gravação foi analisada com a ferramenta `jfr` do JDK 25.

I/O. Nenhum evento `jdk.FileRead` foi registrado durante o clustering. O arquivo CSV é lido apenas na fase de inicialização; as iterações operam exclusivamente sobre o array em memória.

GC. Foram realizadas 60 coletas (51 Young G1New + 9 Old G1Old) com pausa total de 1.024 ms e pausa média de 17 ms por evento. O heap atingiu pico de 2,7 GB, correspondente ao dataset carregado ($\approx 1,75$ GB) mais overhead de trabalho.

Threads. O pool de virtual threads usa os N carriers do `ForkJoinPool` interno. Chunks terminam em $\approx 100\text{--}200$ ms, sendo facilmente visíveis ao JFR ao contrário dos lotes de 5.000 pontos da v2.

Síntese. O parsing CSV desaparece completamente do perfil de execução. O tempo é gasto inteiramente em computação aritmética (`closestCentroid`, `squaredDistance`) distribuída entre os N workers.

5.2 Implementação em Erlang

5.2.1 Descrição

A implementação concorrente em Erlang segue o modelo de atores da BEAM: processos isolados que se comunicam exclusivamente por troca de mensagens, sem memória compartilhada.

O dataset é carregado uma única vez na memória pelo processo principal, com cada ponto representado como um binário compacto de 304 bytes (`«V1:64/float, V2:64/float, ..., V38:64/float»`). Binários maiores que 64 bytes são armazenados em um heap compartilhado com contagem de referências, não sendo copiados ao serem enviados entre processos — equivalente ao `double[][]` do Java em termos de custo de distribuição.

São criados $N = 16$ processos workers de longa duração (*long-lived*), um por scheduler BEAM disponível. Cada worker recebe seu chunk de pontos uma única vez na inicialização e o mantém em seu heap local. Por iteração:

1. O processo principal envia `{iterate, Centroids, From}` a cada worker.
2. Cada worker percorre seu chunk, calcula o centróide mais próximo com `closest_centroid` e acumula somas e contagens parciais locais.
3. Cada worker envia `{partial, Sums, Counts}` de volta ao principal.
4. O principal coleta N respostas, mescla os parciais e recomputa os centróides.

Os centróides são mantidos como listas de floats (`[float()]`), pois são pequenos ($k \times d \times 8 \text{ bytes} \approx 3 \text{ KB}$) e precisam suportar atualização por divisão elemento a elemento.

5.2.2 Resultados

Condição de Corrida Por construção, o modelo de atores Erlang não possui memória compartilhada entre processos. Cada worker acumula somas e contagens em seu próprio heap, e o processo principal é o único a ler e escrever os centróides. Não há race conditions possíveis, dispensando sincronização explícita.

Avaliação com Microbenchmark Os microbenchmarks foram executados com `timer:tc` sobre OTP 25 (BEAM JIT habilitado), com 1 execução de aquecimento e 3 execuções de medição, todas com $k = 10$ e `maxIter = 5`. O dataset ($\approx 1 \text{ GB}$) foi carregado em memória uma única vez antes das medições.

ID	Benchmark	Erlang concorrente	Unidade
B1	Execução completa ($k=10$, <code>maxIter=5</code>)	106.344,7	ms/op
B3	<code>closest_centroid</code> ($k=10$, 38 dims)	15.976,0	ns/op
B4	<code>squared_distance</code> (38 dims)	1.052,7	ns/op

Table 20: Resultados de microbenchmark para a implementação concorrente em Erlang (16 workers, `timer:tc`).

B1 reflete o custo total de 5 iterações sobre 5,75 M pontos com 16 workers. B3 e B4 isolam a aritmética por ponto: `closest_centroid` leva 15,98 μ s e `squared_distance` 1,05 μ s por chamada.

Avaliação de Profile O profile foi capturado com `erlang:statistics` sobre uma execução completa ($k = 10$, `maxIter=10`, 16 workers, dataset ≈ 1 GB). A execução durou 188,7s de parede, equivalente a $\approx 18,9$ s por iteração.

Schedulers. A utilização média dos schedulers BEAM foi de 45,7%. Os workers ficam ociosos enquanto o processo principal executa `collect_partial` e recomputa centróides serialmente; durante esse intervalo os 16 workers aguardam na caixa de mensagens sem consumir CPU.

GC. Foram realizadas 2.129.920 coletas GC em 188,7s (11.329/s), com 246,5 bilhões de palavras reclamadas ($\approx 1,97$ TB). O volume elevado reflete a alta taxa de alocação por ponto: `update_nth` reconstrói parcialmente a lista de somas a cada ponto processado, gerando lixo proporcional a $k \times d$ por chamada. O GC per-processo da BEAM coleta esse lixo de forma isolada, sem pausas globais, mas com custo acumulado significativo.

Síntese. O gargalo é duplo: aritmética por ponto (`squared_distance/closest_centroid`, 113–165 $\times$ mais lenta que Java) e redução serial (main process coleta parciais sequencialmente, mantendo schedulers ociosos $\approx 54\%$ do tempo).

5.3 Discussão

Java v2 $\rightarrow$ v3: eliminação do gargalo de I/O. Nas versões v2, o parsing CSV consumia 86–99% do CPU em cada iteração. Na v3, nenhum evento de leitura de arquivo é registrado durante o clustering. A Tabela 21 resume o impacto.

Abordagem	B1 (s/op)	B2 (s/op)	Erro B1	Erro B2
Serial	212,57	60,05	$\pm 34,48$	$\pm 2,46$
v2-virtual	211,64	71,61	$\pm 260,68$	$\pm 227,05$
v3	4,55	1,60	$\pm 17,98$	$\pm 0,17$

Table 21: Comparativo JMH: serial, v2-virtual e v3 (s/op).

B2 cai de 60 s para 1,6 s (37 $\times$): o custo dominante deixa de ser I/O e passa a ser computação aritmética pura. A variância de B1 colapsa de ± 260 s (v2-virtual) para ± 18 s (v3): sem I/O, cada iteração tem carga uniforme e previsível. No macrobenchmark, o throughput com 1 thread JMeter salta de 1,62 para 52,5 exec/min (32 $\times$); com 16 threads o ganho cai para 14 $\times$ porque os processos concorrentes competem pelos mesmos núcleos. Como trade-off, o heap pico cresce de ≈ 250 MB (v2) para $\approx 2,7$ GB (v3), porém o número de coletas GC cai de 1.000+ para 60, pois há muito menos alocação transiente por iteração.

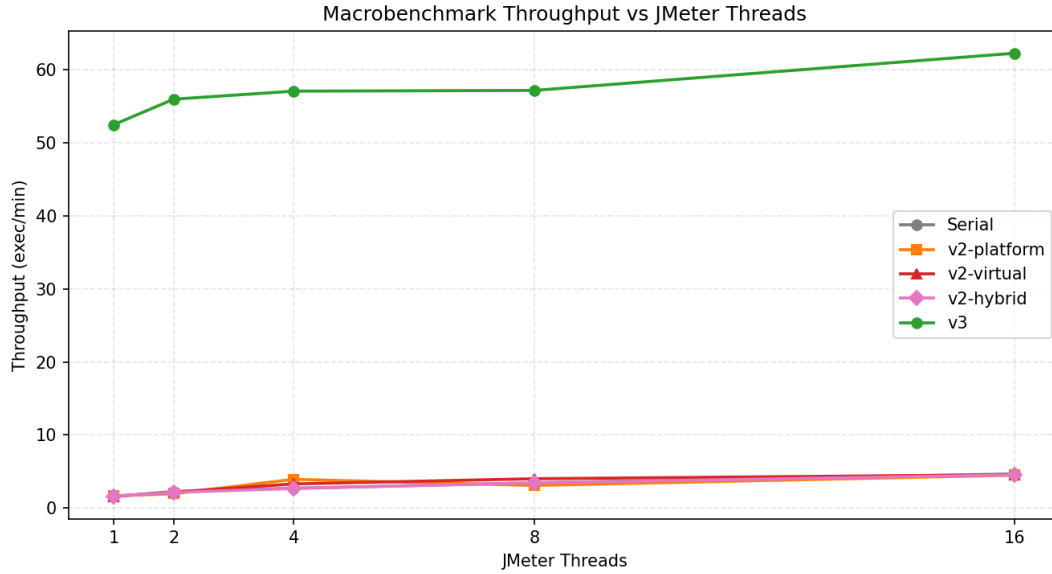


Figure 12: Throughput por número de threads JMeter: painel superior — Serial e variantes v2 com G1GC (escala 0–7 exec/min); painel inferior — v3 com ParallelGC (escala 0–70 exec/min). A diferença de escala evidencia o salto de 32× obtido ao eliminar o parsing CSV das iterações.

Java v3 vs. Erlang: modelo de memória vs. modelo de atores. Com o I/O eliminado em ambas as implementações, a comparação entre Java v3 e Erlang torna-se justa. A Tabela 22 expõe a diferença.

ID	Java v3	Erlang	Fator ($\times$ mais lento)	Un.
B1/B2 (maxIter=5)	1.600,0	106.344,7	66,5	ms/op
B3	96,82	15.976,0	165	ns/op
B4	9,31	1.052,7	113	ns/op

Table 22: Comparativo Java v3 vs. Erlang concorrente (ambos em memória, 16 workers).

A diferença de 66,5× no tempo total é explicada principalmente pela aritmética por ponto (B3/B4): o JIT da JVM otimiza loops sobre arrays de doubles com vetorização SIMD, enquanto a BEAM processa cada valor individualmente sem essa otimização.

Além da aritmética, dois fatores estruturais amplificam a diferença. Primeiro, os schedulers BEAM ficaram com apenas 45,7% de utilização: enquanto o processo principal coleta os resultados parciais e recomputa os centróides de forma sequencial, os 16 workers ficam ociosos aguardando mensagens. Em Java v3, todos os núcleos permanecem ativos até o fim de cada iteração. Segundo, o GC Erlang realizou 2,13 milhões de coletas contra apenas 60 na JVM: a cada ponto processado, a implementação Erlang reconstrói parcialmente a lista de somas, gerando muitos objetos temporários — custo inexistente em Java, que acumula em arrays reutilizáveis.

Quando cada modelo vence. O K-Means é uma carga de trabalho CPU-bound com comunicação mínima entre iterações — o caso em que memória compartilhada tem vantagem estrutural sobre troca de mensagens. O modelo de atores da Erlang traz isolamento e tolerância a falhas sem sincronização explícita, o que é valioso em sistemas distribuídos e de alta disponibilidade, mas não neste workload numérico intensivo.

6 Concorrência Estruturada

6.1 Implementação em Java

6.1.1 Descrição

A v4 parte da v3 — a variante com melhor desempenho até então — e não altera o algoritmo nem o padrão de acumulação local por tarefa. A mudança é inteiramente na camada de coordenação: o par `ExecutorService` + `CountDownLatch` da v3 é substituído por `StructuredTaskScope` (JEP 480) e o snapshot de centróides, antes passado como parâmetro através de `processChunk`, passa a ser propagado via `ScopedValue` (JEP 446). Ambas são APIs de *preview* no JDK 25, exigindo a flag `-enable-preview` em toda invocação da JVM que carregue `KMeans.class` — compilação, JMH, JCStress, JMeter e execução direta.

Essa troca não é uma otimização de desempenho: é uma correção de robustez. Na v3, uma exceção lançada dentro de uma tarefa de worker não tinha canal de propagação de volta à thread coordenadora — o `CountDownLatch` apenas conta conclusões, sem carregar erros. Na prática, isso significa que uma falha real ficaria silenciosamente mascarada, manifestando-se apenas de forma indireta mais adiante (por exemplo, como um resultado ausente na redução). O `StructuredTaskScope` resolve isso estruturalmente: `scope.join()` propaga a exceção original de qualquer subtarefa para a thread coordenadora e cancela automaticamente as subtarefas irmãs ainda em execução — sem necessidade de código adicional de tratamento de erro.

Componente	Alteração
<code>pom.xml</code>	flag <code>-enable-preview</code> adicionada ao <code>maven-compiler-plugin</code>
<code>KMeans.java</code>	<code>ExecutorService</code> + <code>CountDownLatch</code> substituídos por <code>StructuredTaskScope</code> ; centróides propagados por <code>ScopedValue<double[][]> CENTROIDS</code> em vez de parâmetro de método
<code>justfile</code>	flag <code>-enable-preview</code> adicionada a todas as invocações de <code>java/JMH/JCStress/JMeter</code>

Table 23: Alterações da v4 em relação à v3.

O confinamento local por tarefa (`localSums/localCounts` dentro de `processChunk`) é mantido exatamente como na v2/v3: a v4 contribui apenas na camada de coordenação, não na camada de confinamento, que já havia sido validada por JCStress nas versões anteriores.

6.1.2 Resultados

Avaliação de Condição de Corrida Os mesmos quatro testes JCStress (JCS1–JCS4) das seções anteriores foram executados sobre a v4, com a mesma configuração (`preset default`, 6 forks, 5 iterações de 1.000 ms por fork).

ID	Resultado	Outcome FORBIDDEN	Freq. máx.
JCS1	PASSOU	nenhum	—
JCS2	PASSOU	nenhum	—
JCS3	FALHOU (esperado)	"1" lost update em <code>counts[0]++</code>	30,00%
JCS4	FALHOU (esperado)	"1.0" lost update em <code>sums[0]+=1.0</code>	29,61%

Table 24: Resultados JCSstress — regiões críticas da v4.

JCS1 e JCS2 continuam passando: a leitura concorrente do centróide via `ScopedValue` é segura, assim como a leitura direta por parâmetro nas versões anteriores. JCS3 e JCS4 falham como esperado — o teste exercita deliberadamente o mesmo padrão de array compartilhado sem proteção, independente da camada de coordenação usada. A frequência máxima observada (30,00% e 29,61%) está na mesma ordem de grandeza da v2 (28,31%/27,10%); a frequência exata de um outcome de *lost update* é inerentemente dependente do escalonamento de threads no momento da execução, não um valor que deveria se repetir entre execuções da mesma condição de corrida.

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1, com `-enable-preview`, 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada.

ID	Benchmark		v3	v4	Erro v4 ($\pm$)	Un.
B1	Execução completa ($k=10$, $\text{iter}=20$)		4,55	7,08	1,26	s/op
B2	Por iteração ($k=10$, $\text{iter}=5$)		1,60	1,98	0,07	s/op
B3	<code>closestCentroid</code> (38 dims)	96,82	131,18		97,71	ns/op
B4	<code>squaredDistance</code> (38 dims)	9,31	12,89		6,79	ns/op

Table 25: Resultados JMH — v3 vs. v4.

Todos os quatro benchmarks pioram levemente em relação à v3: B2 sobe de 1,60 s para 1,98 s (~24% mais lento), e B3/B4 — que medem exatamente o mesmo código aritmético de `closestCentroid` e `squaredDistance`, inalterado desde a v1 — também aumentam, embora com erro elevado. Como o algoritmo em si não mudou, esse custo adicional é atribuído à sobrecarga das APIs de *preview*: tanto `StructuredTaskScope` quanto `ScopedValue` ainda não têm a maturidade de compilação JIT do caminho `ExecutorService+CountDownLatch`, que é parte estável da plataforma há muito mais tempo. Este é um resultado honesto a relatar, não um problema a esconder: a v4 troca uma pequena perda de desempenho por uma coordenação estruturalmente mais robusta.

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 ($k=10$, $\text{maxIter}=3$, $\text{tol}=0,0$), 120 s por configuração e ramp-up de 30 s. Apenas o coletor **ParallelGC** foi testado — decisão de escopo para a v4 (e v5), sem repetir a varredura de três coletores feita em v1–v3.

Threads	Execuções	Tempo (s)	Throughput (exec/min)
1	92	120,4	45,8
2	97	121,3	48,0
4	99	123,0	48,3
8	101	124,4	48,7
16	115	132,7	52,0

Table 26: Throughput (execuções/min, ParallelGC) por número de threads JMeter — v4.

Comparado à v3 com o mesmo coletor (52,5 a 62,3 exec/min de 1 a 16 threads), a v4 fica abaixo em todos os níveis de carga (45,8 a 52,0 exec/min) — consistente com o aumento já observado em B2: o custo adicional por iteração se propaga para o throughput agregado.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo. A gravação totalizou 1.806 amostras de `jdk.ExecutionSample`.

Método	Amostras	CPU (%)	Categoria
<code>KMeans.squaredDistance</code>	588	32,6	Algoritmo
<code>String.split</code>	181	10,0	Parsing CSV (inicialização)
<code>FloatingDecimal.copyDigits</code>	162	9,0	Parsing CSV (inicialização)
<code>Arrays.copyOf</code>	159	8,8	Coordenação (subtarefas)
<code>KMeans.closestCentroid</code>	157	8,7	Algoritmo
<code>FloatingDecimal.readJavaFormatString</code>	125	6,9	Parsing CSV (inicialização)
<code>KMeans.addPoint</code>	92	5,1	Algoritmo
<code>FloatingDecimal.doubleValue</code>	75	4,2	Parsing CSV (inicialização)
<code>BufferedReader.readLine</code>	26	1,4	I/O

Table 27: Hot methods na v4 (JFR, 1.806 amostras).

O algoritmo (`squaredDistance` + `closestCentroid` + `addPoint`) soma ~46% das amostras, o maior bloco individual. Chama atenção a persistência de frames de *parsing* de CSV (`String.split`, `FloatingDecimal.*`) somando ~39%: isso não é um retorno do gargalo de I/O por iteração eliminado na v3 — é a leitura única do dataset (`loadAllPoints`, executada uma vez antes do laço de iterações) caindo dentro da janela de gravação do JFR, mesmo estando excluída da medição de tempo do JMH/JMeter. O bloco `Arrays.copyOf` (~9%) é overhead de coordenação — a lista de `StructuredTaskScope.Subtask` sendo montada e convertida a cada iteração.

Foram registradas 61 coletas de GC, com pausa total de 1.013,97 ms e pausa média de 16,62 ms por evento. O heap atingiu pico de 2.764,8 MB (~2,76 GB), em linha com o ~2,7 GB da v3 — esperado, já que ambas mantêm o mesmo dataset em memória como `double[][]`. GC não é gargalo em nenhuma das duas versões.

6.2 Discussão

A Tabela 28 resume a evolução do custo por iteração desde a versão serial.

Abordagem	B1 (s/op)	B2 (s/op)	Erro B1	Erro B2
Serial	212,57	60,05	$\pm 34,48$	$\pm 2,46$
v3	4,55	1,60	$\pm 17,98$	$\pm 0,17$
v4	7,08	1,98	$\pm 1,26$	$\pm 0,07$

Table 28: Comparativo JMH: Serial, v3 e v4 (s/op).

A v4 não é uma otimização de desempenho — é uma atualização de idioma de concorrência. A v3 tinha uma lacuna estrutural real: por usar `ExecutorService+CountDownLatch`, uma exceção lançada dentro de uma tarefa de worker não possuía canal de propagação de volta à thread coordenadora, podendo ficar silenciosamente mascarada. O `StructuredTaskScope` fecha essa lacuna por construção: `scope.join()` propaga a exceção original e cancela automaticamente as subtarefas irmãs. O `ScopedValue` substitui a passagem do snapshot de centróides por parâmetro de método, sendo a alternativa moderna ao `ThreadLocal` — que não se propaga corretamente para virtual threads filhas criadas dentro do escopo.

O preço dessa robustez é um aumento modesto no tempo por iteração (B2: 1,60 s $\rightarrow$ 1,98 s) e no throughput agregado (52,5–62,3 $\rightarrow$ 45,8–52,0 exec/min), atribuído à imaturidade de compilação das duas APIs de *preview*. O JCSstress confirma que o confinamento local por tarefa — a garantia que realmente importa para corretude — permanece inalterado: JCS1/JCS2 continuam passando, e JCS3/JCS4 continuam falhando exatamente como esperado quando a proteção é removida deliberadamente.

7 Fork/Join — Divisão e Conquista

7.1 Implementação em Java

7.1.1 Descrição

A v3 e a v4 particionam o array de pontos em exatamente N fatias fixas (uma por processador lógico) antes do início do laço de iterações. A v5 substitui esse particionamento fixo por divisão-e-conquista recursiva com `RecursiveTask<PartialResult>`: cada tarefa recebe um intervalo `[from, to)` do array e, se o intervalo for maior que um limiar, divide-o ao meio, bifurca (`fork()`) a metade esquerda para execução assíncrona, computa a metade direita diretamente, e por fim junta (`join()`) o resultado da esquerda para mesclar os dois parciais. Esse padrão traz *work-stealing* para o nível da tarefa: threads ociosas do `ForkJoinPool` podem roubar subtarefas pendentes de outras threads, o que é útil quando a carga de trabalho não está perfeitamente balanceada entre partições — não é o caso deste dataset sintético e uniforme, mas é um modelo estruturalmente diferente do particionamento estático da v3/v4, e por isso vale a pena explorar mesmo sem garantia de ganho de desempenho. O `ScopedValue<double[] []> CENTROIDS` introduzido na v4 é mantido sem alterações, já que se propaga corretamente para subtarefas bifurcadas a partir de um escopo `ScopedValue.where(...).call(...)`.

O limiar de divisão é calculado como:

```
int n = Runtime.getRuntime().availableProcessors();
int threshold = points.length / (n * 4);
```

ou seja, o alvo inicial é de 4 subtarefas por núcleo — confirmado diretamente no código-fonte da tag v5. Não foi realizado teste de sensibilidade variando esse limiar: o valor é reportado como uma escolha de design (*work-stealing* só é útil se houver subtarefas suficientes para roubar) e não como um parâmetro ajustado empiricamente.

Componente	Alteração
KMeans.java	particionamento fixo em N chunks substituído por <code>KMeansTask extends RecursiveTask<PartialResult></code> (divisão recursiva por limiar, <code>fork()/join()</code>); <code>ForkJoinPool.commonPool()</code> no lugar do <code>ExecutorService</code> dedicado

Table 29: Alterações da v5 em relação à v4.

7.1.2 Resultados

Avaliação de Condição de Corrida Os mesmos quatro testes JCSstress (JCS1–JCS4) foram executados sobre a v5, com a mesma configuração das seções anteriores.

ID	Resultado	Outcome FORBIDDEN	Freq. máx.
JCS1	PASSOU	nenhum	—
JCS2	PASSOU	nenhum	—
JCS3	FALHOU (esperado)	"1" lost update em <code>counts[0]++</code>	28,20%
JCS4	FALHOU (esperado)	"1.0" lost update em <code>sums[0]+=1.0</code>	28,96%

Table 30: Resultados JCSstress — regiões críticas da v5.

O padrão se repete: JCS1/JCS2 passam (leitura concorrente segura), JCS3/JCS4 falham como esperado, na mesma ordem de grandeza das versões anteriores. O modelo de divisão recursiva não introduz nenhuma nova região crítica — cada subtarefa folha ainda acumula em arrays locais próprios, e a mesclagem de parciais só ocorre depois que `join()` garante a conclusão da subtarefa filha, preservando o mesmo padrão de confinamento local com redução sequencial das versões anteriores.

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1, com `-enable-preview` (herdado da v4 para o `ScopedValue`), 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada.

ID	Benchmark	v4	v5	Erro v5 ($\pm$)	Un.
B1	Execução completa ($k=10$, iter=20)	7,08	5,72	16,08	s/op
B2	Por iteração ($k=10$, iter=5)	1,98	1,78	0,14	s/op
B3	<code>closestCentroid</code> (38 dims)	131,18	115,62	20,77	ns/op
B4	<code>squaredDistance</code> (38 dims)	12,89	11,10	1,81	ns/op

Table 31: Resultados JMH — v4 vs. v5.

B2 melhora levemente frente à v4 (1,98 s $\rightarrow$ 1,78 s), mas o dado mais chamativo é a variância de B1: $\pm 16,08$ s, a maior de toda a série desde a v2-virtual ($\pm 260,68$ s). O padrão é semelhante ao já observado naquela versão — tarefas CPU-bound que nunca bloqueiam introduzem ruído de escalonamento quando expostas a um pool pensado originalmente para *work-stealing* de tarefas que podem ceder o processador. Aqui o efeito é bem menor em magnitude, mas aponta na mesma direção: a divisão recursiva com `fork()/join()` tem overhead de agendamento que o particionamento estático fixo da v3/v4 não tem.

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 ($k = 10$, `maxIter=3`, `tol=0,0`), 120 s por configuração e ramp-up de 30 s, com **ParallelGC** (mesma decisão de escopo da v4 — sem varredura de coletores para v5).

Threads	Execuções	Tempo (s)	Throughput (exec/min)
1	93	120,2	46,4
2	99	120,6	49,3
4	100	122,2	49,1
8	102	123,8	49,4
16	112	128,3	52,4

Table 32: Throughput (execuções/min, ParallelGC) por número de threads JMeter — v5.

O throughput da v5 é muito próximo ao da v4 em todos os níveis de carga (faixa de 45,8–52,0 na v4 contra 46,4–52,4 na v5), ambos abaixo da v3 (52,5–62,3). Isso reforça a leitura da seção anterior: o custo adicional de coordenação — seja via **StructuredTaskScope** (v4) ou via **ForkJoinPool** com divisão recursiva (v5) — supera, neste dataset uniforme e de carga bem distribuída, qualquer ganho estrutural que *work-stealing* poderia oferecer.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo. A gravação totalizou 2.452 amostras de `jdk.ExecutionSample`.

Método	Amostras	CPU (%)	Categoria
<code>KMeans.squaredDistance</code>	1084	44,2	Algoritmo
<code>KMeans.closestCentroid</code>	253	10,3	Algoritmo
<code>FloatingDecimal.copyDigits</code>	193	7,9	Parsing CSV (inicialização)
<code>String.split</code>	185	7,5	Parsing CSV (inicialização)
<code>FloatingDecimal.parseDouble</code>	177	7,2	Parsing CSV (inicialização)
<code>FloatingDecimal.readJavaFormatString</code>	138	5,6	Parsing CSV (inicialização)
<code>FloatingDecimal.doubleValue</code>	78	3,2	Parsing CSV (inicialização)
<code>BufferedReader.readLine</code>	51	2,1	I/O

Table 33: Hot methods na v5 (JFR, 2.452 amostras).

O algoritmo (`squaredDistance` + `closestCentroid`) domina com ~55% das amostras — proporção maior que na v4 (~46%), sinal de que a divisão recursiva tem menos overhead de coordenação visível no profile que o `StructuredTaskScope`. O *parsing* de CSV da leitura única inicial (`loadAllPoints`) segue presente pelo mesmo motivo já discutido na v4: a janela de gravação do JFR cobre a inicialização, mesmo que ela seja excluída da medição JMH/JMeter. O arquivo `io.txt` para a v5 não registra nenhum evento `jdk.FileRead`, diferente da v4 (que tinha 17 linhas de eventos de *classloading*, sem leitura de dataset em nenhuma das duas).

Foram registradas 56 coletas de GC, com pausa total de 978,19 ms e pausa média de 17,47 ms por evento. O heap atingiu pico de 2.560,0 MB (~2,56 GB), próximo dos ~2,7 GB e ~2,76 GB de v3 e v4 — esperado, pelo mesmo dataset em memória.

7.2 Discussão

A Tabela 34 estende o comparativo com a v5.

Abordagem	B1 (s/op)	B2 (s/op)	Erro B1	Erro B2
Serial	212,57	60,05	±34,48	±2,46
v3	4,55	1,60	±17,98	±0,17
v4	7,08	1,98	±1,26	±0,07
v5	5,72	1,78	±16,08	±0,14

Table 34: Comparativo JMH: Serial, v3, v4 e v5 (s/op).

A v5 é, como a v4, uma variação estrutural na camada de coordenação, não uma otimização de desempenho: os números de B2 e de throughput ficam muito próximos aos da v4, ambos abaixo da v3. A diferença pedagógica está no modelo — divisão-e-conquista recursiva com *work-stealing* em vez de particionamento estático — que só demonstraria vantagem real em cargas desbalanceadas entre partições, o que não é o caso

deste dataset sintético e uniformemente distribuído. O JCSstress confirma, mais uma vez, que o confinamento local por tarefa permanece correto independentemente de como as tarefas são criadas e distribuídas: JCS1/JCS2 passam, JCS3/JCS4 falham exatamente como esperado.

A variância elevada de B1 ($\pm 16,08$ s) é o achado mais interessante desta seção: mesmo em uma escala bem menor que a da v2-virtual ($\pm 260,68$ s), ela reforça a mesma lição — pools de tarefas pensados para *work-stealing* introduzem ruído de escalonamento em cargas CPU-bound que nunca bloqueiam, e esse efeito aparece tanto em virtual threads sobre ForkJoinPool (v2-virtual) quanto em divisão recursiva explícita sobre o mesmo framework (v5).

8 Spark — Processamento Distribuído

8.1 Implementação em Java

8.1.1 Descrição

Todas as versões anteriores (v2–v5) compartilham memória dentro de uma única JVM e coordenam threads sobre um mesmo `double[] []`. A v6 substitui esse modelo por Apache Spark 4.0 em modo `local[*]`: o dataset passa a viver como um `Dataset<Row>/RDD` particionado, gerenciado internamente pelo Spark, e cada iteração do algoritmo é expressa como uma transformação distribuída em vez de um laço explícito sobre arrays. Por exigir Spark 4.0, que é testado sobre JDK 21 e não sobre o JDK 25 usado em v1–v5, esta versão roda em um JDK diferente de todo o restante do trabalho.

Por iteração, cada ponto é mapeado para um par (`cluster`, `double[d+1]`) — onde os primeiros d elementos são as features do ponto e o último é a contagem (sempre 1,0) — via `mapToPair`, seguido de um `reduceByKey` que soma localmente por partição antes de mesclar entre partições. É exatamente o mesmo padrão de acumulação local usado por `processChunk` nas versões v3–v5, agora expresso em operações de RDD em vez de arrays locais explícitos. O snapshot dos centróides a cada iteração é distribuído a todos os executores via `Broadcast<double[] []>`, ocupando o mesmo papel que o `ScopedValue/copy()` ocupava nas versões anteriores.

Foram testadas duas variantes de formato de armazenamento sobre o mesmo dataset (5,75 milhões de pontos, 38 features): uma lendo o CSV original e outra lendo uma conversão Parquet do mesmo conteúdo. As tags correspondentes são `v6-csv` e `v6-parquet`.

Componente	Alteração
<code>pom.xml</code>	<code>maven.compiler.release</code> alterado de 25 para 21; dependências de JMH, JCTest e JMeter removidas; adicionados <code>spark-core_2.13</code> e <code>spark-sql_2.13</code> 4.0.0
<code>KMeans.java</code>	reescrita completa: <code>SparkSession local[*]</code> , carga do dataset via <code>DataFrame</code> , <code>mapToPair</code> + <code>reduceByKey</code> por iteração, <code>Broadcast</code> do snapshot de centróides
<code>Main.java</code>	remove o uso de <code>CsvDataset</code> ; cronometragem por <code>System.nanoTime()</code> em torno da execução completa

Table 35: Alterações da v6 em relação à v5.

A Figura 13 mostra o laço de iteração completo: o broadcast do snapshot de centróides seguido do par `mapToPair+reduceByKey` que acumula somas e contagens por cluster, o mesmo padrão de acumulação local das versões anteriores expresso em operações de RDD.

```

for (int iter = 1; iter <= maxIterations; iter++) {
    final double[][] snapshot = copy(centroids);
    // Broadcast centroid snapshot to all partitions – same role as the
    // defensive copy() in v3/v4/v5: prevents reading partially-updated state
    Broadcast<double[][]> bcCentroids = sc.broadcast(snapshot);
    final int dims = d;

    // Each point becomes (cluster, [f0..fd-1, count=1.0]).
    // reduceByKey sums partition-locally then merges across partitions –
    // the same local-accumulation pattern as processChunk in v3/v4/v5
    Map<Integer, double[]> clusterSums = points
        .mapToPair(point -> {
            int cluster = closestCentroid(point, bcCentroids.value());
            double[] acc = Arrays.copyOf(point, dims + 1);
            acc[dims] = 1.0; // count in last element
            return new Tuple2<>(cluster, acc);
        })
        .reduceByKey((a, b) -> {
            double[] merged = new double[dims + 1];
            for (int i = 0; i <= dims; i++) merged[i] = a[i] + b[i];
            return merged;
        })
        .collectAsMap();

    bcCentroids.destroy();
}

```

Figure 13: Broadcast do snapshot de centróides e acumulação por `mapToPair+reduceByKey` a cada iteração (`KMeans.cluster`, tag `v6-csv`).

A Figura 14 mostra a única diferença de código entre as duas variantes testadas: o formato de leitura do `DataFrame`. Todo o restante do arquivo — `broadcast`, `mapToPair`, `reduceByKey`, avaliação final — é idêntico entre `v6-csv` e `v6-parquet`.

```

// v6-csv
Dataset<Row> df = spark.read()
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(datasetPath)
    .drop(NON_FEATURE_COLUMNS)
    .cache();

// v6-parquet
Dataset<Row> df = spark.read()
    .parquet(datasetPath)
    .drop(NON_FEATURE_COLUMNS)
    .cache();

```

Figure 14: Única diferença de código entre `v6-csv` e `v6-parquet`: o formato de leitura do `DataFrame` (`KMeans.cluster`).

O diagrama de classes da Figura 15 mostra a estrutura da `v6`, bem mais simples que a das versões anteriores: `Main` chama `KMeans.cluster`, que devolve um `KMeansResult` (o mesmo *record* usado desde a `v1`) e depende diretamente de três tipos da API do Spark — `SparkSession`, `Dataset<Row>` e `Broadcast<double[][]>`. O `CsvDataset` das versões anteriores desaparece do grafo de dependências: o arquivo ainda existe fisicamente no repositório da tag `v6-csv`, mas não é mais referenciado em nenhum ponto do código.

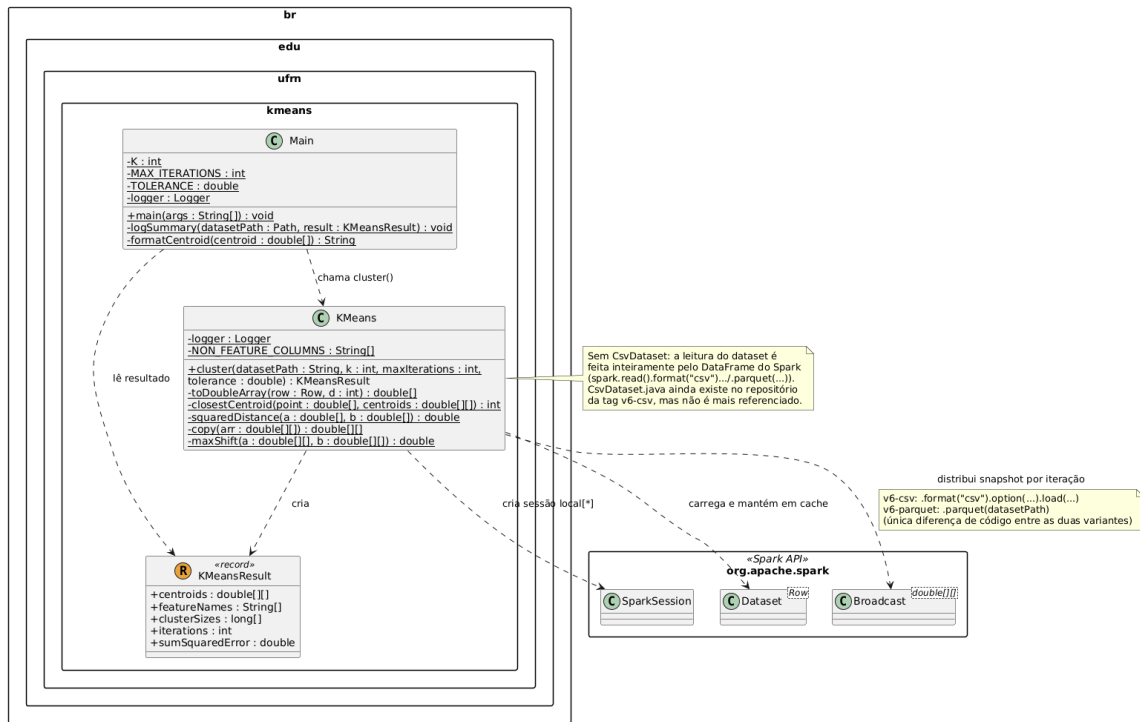


Figure 15: Diagrama de classes da v6 (Spark). `Main` e `KMeans` seguem estáticas como nas versões anteriores; `KMeansResult` é reaproveitado sem alterações. A única diferença de código entre `v6-csv` e `v6-parquet` está no carregamento do `Dataset<Row>` (ver Figura 14).

8.1.2 Resultados

Aplicabilidade das Ferramentas de Benchmark Nenhuma das ferramentas usadas em `v1-v5` se aplica diretamente à `v6`. `JMH` não é viável porque a inicialização de uma `SparkSession` é pesada demais para caber no modelo de fork/iterações curtas do `JMH`. `JMeter` não se aplica porque o Spark não foi projetado para atender chamadas concorrentes de alta frequência — rodar `JMeter` contra uma única sessão Spark compartilhada testaria o escalonador do Spark, não o algoritmo. `JCStress` não se aplica porque o modelo de execução do Spark não tem estado mutável compartilhado entre tarefas por construção — não há região crítica para testar. As métricas usadas nesta versão são **tempo de parede** (`System.nanoTime()` em torno da execução completa) e **profile JFR**, complementado pela Spark UI (<http://localhost:4040>) para inspeção do DAG de jobs e stages.

Avaliação de Tempo de Parede Ambas as variantes foram executadas com $k = 10$ e `maxIter`=10 (sem convergência dentro desse limite), sobre o mesmo dataset de 5,75 milhões de pontos e 38 features.

Variante	Tempo total (ms)	Tempo/iter.* (s)
v6-csv	34.624	3,46
v6-parquet	44.264	4,43

Table 36: Tempo de parede — v6-csv vs. v6-parquet ($k = 10$, `maxIter`=10, mesmo dataset de 5,75 milhões de pontos; SSE final praticamente idêntico entre as duas variantes, ver texto). * Estimado como tempo total $\div$ 10 iterações; não é uma medição isolada como o B2 do JMH nas versões anteriores, apenas uma referência aproximada de ordem de grandeza.

O SSE final é praticamente idêntico entre as duas variantes — confirmando que ambas processam o mesmo dataset e chegam ao mesmo resultado, como esperado, já que a única diferença de código entre elas é o formato de leitura do `DataFrame`. Contra a expectativa comum de que um formato colunar e comprimido como o Parquet seria mais rápido de ler que texto puro, o **Parquet ficou ~28% mais lento que o CSV** nesta carga (44.264 ms contra 34.624 ms). A seção de profile a seguir explica a causa.

Avaliação de Profile v6-csv. A gravação JFR totalizou 11.263 amostras de execução.

Método	Amostras	CPU (%)	Categoria
<code>KMeans.squaredDistance</code>	1475	13,1	Algoritmo
<code>CharInputReader.getString</code>	1179	10,5	Parsing CSV (univocity)
<code>CompressibleColumnBuilder</code>	621	5,5	Cache colunar (Spark)
<code>HashMap.put0</code>	413	3,7	Overhead Scala/Spark
<code>KMeans.closestCentroid</code>	352	3,1	Algoritmo
<code>LineReader.readDefaultLine</code>	330	2,9	Parsing CSV (Hadoop)

Table 37: Hot methods na v6-csv (JFR, 11.263 amostras).

Diferente da v3 — onde nenhum evento de leitura de dataset aparece depois da carga inicial —, os frames de *parsing* de CSV (`AbstractCharInputReader`, `LineReader`) seguem entre os mais amostrados mesmo depois da carga inicial. O motivo aparece na saída de execução: o Spark registra repetidos avisos `MemoryStore - Not enough space to cache rdd_X in memory!` — sob os 4 GB de memória do driver, nem todas as partições do dataset cabem no cache, e as partições que não couberam são **relidas e reparseadas do CSV a cada iteração** em vez de servidas da memória. Isso é uma diferença estrutural importante frente à v3: lá, o dataset ou cabe inteiro no heap ou a JVM falha; aqui, o cache do Spark degrada silenciosamente para recomputação sob pressão de memória, sem erro explícito.

v6-parquet. A gravação JFR totalizou 11.016 amostras de execução.

Método	Amostras	CPU (%)	Categoria
KMeans.squaredDistance	2152	19,5	Algoritmo
ParquetDictionary.decodeToLong	912	8,3	Decodificação Parquet
CodegenIterator.processNext	813	7,4	Codegen (Spark)
CompressibleColumnBuilder	757	6,9	Cache colunar (Spark)
HashMap.findNode	659	6,0	Overhead Scala/Spark
KMeans.closestCentroid	603	5,5	Algoritmo
ParquetDictionary.decodeToDouble	442	4,0	Decodificação Parquet

Table 38: Hot methods na v6-parquet (JFR, 11.016 amostras).

Os mesmos avisos de pressão de memória aparecem no log do v6-parquet, mas com um comportamento diferente: em vez de descartar o bloco que não coube, o **BlockManager** registra "**Persisting block to disk instead**" — o formato colunar e comprimido do Parquet muda a forma como o Spark decide lidar com a falta de espaço no cache. Ainda assim, o tempo total foi maior: a decodificação de dicionário/delta do Parquet (**ParquetDictionary.decodeToLong/decodeToDouble**) soma uma fração relevante das amostras, um custo de CPU real que o CSV — apesar de mais lento para ler bruto — não paga da mesma forma depois que uma partição está em cache.

Heap e GC. O pico de heap foi de 3.891,2 MB (~3,89 GB) na v6-csv e 3.993,6 MB (~3,99 GB) na v6-parquet — ambos notavelmente acima dos ~2,6–2,76 GB de v3/v4/v5, apesar de operarem sobre o mesmo dataset bruto: as estruturas internas do Spark (representação colunar em cache, overhead de execução distribuída no driver e nos executores locais) somam memória real acima dos dados em si, e ambas as variantes ficam próximas do teto configurado de 4 GB (**-Xmx4g**) — consistente com os avisos de pressão de cache já discutidos. A v6-csv registrou 243 coletas de GC e a v6-parquet 124; esses números não são diretamente comparáveis aos das versões anteriores, pois incluem sobrecarga do próprio Spark/JVM além do algoritmo.

8.2 Discussão

O Spark é, neste dataset e nesta escala, **mais lento que v3/v4/v5**: o tempo por iteração aproximado (3,46 s na v6-csv, 4,43 s na v6-parquet) fica bem acima do B2 de v3/v4/v5 (1,60–1,98 s). O dataset inteiro cabe confortavelmente em memória numa única máquina, e o custo de serialização, broadcast e gerenciamento de partições distribuídas do Spark supera qualquer benefício que o processamento distribuído ofereceria numa escala maior. O valor pedagógico desta versão está na mudança radical de modelo de programação — de threads coordenadas sobre memória compartilhada para transformações distribuídas sobre um **Dataset** particionado — não em um ganho de desempenho.

A comparação entre CSV e Parquet dentro da própria v6 é o achado mais interessante desta seção: contra a intuição comum de que um formato colunar e comprimido seria mais rápido, o **Parquet foi ~28% mais lento** nesta carga, porque o custo de decodificação de dicionário/delta por linha supera a economia de I/O bruto. É o mesmo tipo de lição que o profile da versão serial já havia ensinado — **medir antes de assumir** — agora aplicada à escolha de formato de armazenamento em vez de à escolha de estratégia de

concorrência.

Por fim, a v6 expõe um modo de falha que nenhuma versão single-JVM tem: sob pressão de memória, o cache do Spark degrada silenciosamente para recomputação (re-lendo/reparseando partições, como visto na v6-csv) em vez de falhar de forma explícita. Na v3, o dataset ou cabe inteiro no heap ou a JVM lança `OutOfMemoryError` — uma falha visível e imediata. No Spark, a mesma restrição de memória se manifesta como uma degradação de desempenho silenciosa, que só fica evidente ao inspecionar o profile ou os logs de aviso do `MemoryStore`.

9 Discussão Final

9.1 Comparativo Geral das Abordagens

O trabalho evoluiu em seis estágios principais do lado Java (serial, v2, v3, v4, v5, v6), cada um respondendo a uma limitação do estágio anterior, além de uma implementação paralela em Erlang usada como ponto de comparação de modelo de concorrência.

Serial. A implementação sequencial estabeleceu a baseline: uma thread lê o CSV, calcula a distância de cada ponto aos centróides e acumula os resultados. O perfil de execução revelou que $\approx 82\%$ do tempo era gasto em parsing CSV (`FloatingDecimal.parseDouble`, `String.split`), e apenas $\approx 8\%$ em computação aritmética. Isso significa que o algoritmo em si é rápido; o gargalo é a entrada de dados.

Concorrente v2 (platform, virtual, hybrid). A paralelização da fase de atribuição (lotes de 5.000 pontos, pool de N workers) não trouxe ganho de tempo por iteração: o parsing CSV seguia sequencial na thread principal, dominando o tempo total. Os microbenchmarks mostraram B2 ≈ 60 s/op para todas as variantes, idêntico ao serial. O macrobenchmark revelou throughput de 1,6–4,6 exec/min dependendo do número de threads JMeter e da variante de GC.

A exploração das três variantes foi útil para entender o comportamento de cada modelo de thread. A variante hybrid foi a mais reveladora: ao usar uma virtual thread como coordenadora, o parsing CSV ficou invisível ao profiler — porque a thread suspende durante I/O em vez de consumir CPU. Isso confirmou que o gargalo não estava na coordenação dos workers, mas na leitura do arquivo em si.

Concorrente v3 (dados em memória). Carregar o dataset inteiro em `double[][]` antes do loop de iterações eliminou o parsing CSV das medições. B2 caiu de ≈ 60 s para $\approx 1,6$ s ($\approx 37\times$); o throughput saltou de $\approx 1,6$ para ≈ 52 exec/min com 1 thread JMeter. O perfil não registrou nenhum evento de leitura de arquivo durante o clustering. O custo foi o aumento do heap de ≈ 250 MB para $\approx 2,7$ GB.

Concorrente v4 (concorrência estruturada). A v4 substitui o par `ExecutorService` + `CountDownLatch` da v3 por `StructuredTaskScope` e `ScopedValue` (JEP 480/446), corrigindo uma lacuna estrutural real: exceções lançadas dentro de uma tarefa de worker, antes sem canal de propagação de volta à coordenadora, passam a propagar corretamente via `scope.join()`, que também cancela subtarefas irmãs automaticamente. O preço é um aumento modesto no custo por iteração (B2: 1,60 s $\rightarrow$ 1,98 s, $\approx 24\%$) e no throughput (52,5–62,3 $\rightarrow$ 45,8–52,0 exec/min), atribuído à imaturidade de compilação das APIs de *preview*. O confinamento local por tarefa permanece inalterado e validado por JCSstress.

Concorrente v5 (fork/join). A v5 troca o particionamento fixo em N chunks por divisão-e-conquista recursiva com `RecursiveTask` e *work-stealing*. Os números ficam muito próximos aos da v4 (B2 = 1,78 s; throughput 46,4–52,4 exec/min) — o dataset uniforme deste trabalho não oferece o desbalanceamento entre partições que o *work-stealing* precisaria para mostrar vantagem real. O achado mais relevante é a variância elevada de B1 ($\pm 16,08$ s), ecoando o mesmo efeito já visto na v2-virtual: pools de tarefas pensados para trabalho que cede o processador introduzem ruído de escalonamento em cargas puramente aritméticas que nunca bloqueiam.

Spark (v6, processamento distribuído). A v6 abandona o modelo de memória compartilhada em favor de um `Dataset<Row>` particionado gerenciado pelo Spark, com `mapToPair+reduceByKey` por iteração e broadcast do snapshot de centróides. Nem JMH, nem JMeter, nem JCSstress se aplicam a este modelo; a métrica usada foi tempo de parede.

O tempo por iteração aproximado ($\approx 3,46$ s no CSV; $\approx 4,43$ s no Parquet) fica acima do B2 de v3–v5 — o custo de coordenação distribuída supera o benefício em um dataset que cabe confortavelmente em uma única máquina. O achado mais interessante foi contraintuitivo: o Parquet, formato colunar e comprimido, foi $\approx 28\%$ *mais lento* que o CSV bruto nesta carga, pela sobrecarga de decodificação de dicionário/delta por linha. O cache do Spark também expôs um modo de falha ausente nas versões single-JVM: sob pressão de memória, partições não cacheadas são silenciosamente relidas/reparseadas em vez de causar um erro explícito.

Erlang concorrente (modelo de atores, dados em memória). Esta implementação parte do mesmo ponto que a v3 (dados em memória) e não foi estendida para acompanhar v4–v6, que são extensões específicas do lado Java; a comparação a seguir é sempre contra a v3. Distribui chunks entre 16 workers BEAM, sem memória compartilhada. O custo por iteração ficou em $\approx 21,3$ s ($\approx 66\times$ acima do Java v3 — ver Tabela 22). O gargalo é a aritmética por ponto: B3 e B4 são $113\text{--}165\times$ mais lentos que em Java, pois a BEAM desempacota floats de binários sem vetorização SIMD. Os schedulers operaram a $45,7\%$ de utilização — a redução centralizada serializa parte de cada iteração.

A Tabela 39 resume os principais números de cada abordagem.

Abordagem	B2 (s/op)	Throughput 1t	Throughput 16t	Heap pico
Serial	60,05	1,56/min*	4,66/min*	≈ 250 MB
v2-platform	60,69	1,65/min	4,49/min	≈ 250 MB
v2-virtual	71,61	1,62/min	4,55/min	≈ 250 MB
v2-hybrid	60,44	1,60/min	4,53/min	≈ 250 MB
v3	1,60	52,5/min	62,3/min	$\approx 2,7$ GB
v4	1,98	45,8/min	52,0/min	$\approx 2,76$ GB
v5	1,78	46,4/min	52,4/min	$\approx 2,56$ GB
v6-csv	3,46 [‡]	N/A [‡]	N/A [‡]	$\approx 3,89$ GB
v6-parquet	4,43 [‡]	N/A [‡]	N/A [‡]	$\approx 3,99$ GB
Erlang (16 workers)	21,27 [†]	N/A	N/A	$\approx 1,75$ GB

Table 39: Comparativo geral entre abordagens (B2 = custo por iteração; throughput = exec/min, JMeter G1GC exceto v3–v5 com ParallelGC; Erlang sem macrobenchmark HTTP; v6 sem JMH/JMeter, ver texto). *Serial: N instâncias independentes executadas em paralelo pelo JMeter (sem concorrência interna); ver Tabela 7. [†]Erlang B2 estimado como $B1 / \maxIter$ ($106.344,7 \text{ ms} \div 5 \div 1000$); sem benchmark B2 dedicado. [‡]v6: sem JMH dedicado; B2 estimado como tempo de parede total $\div 10$ iterações (ver Tabela 36 na Seção 8); sem macrobenchmark JMeter — Spark não foi projetado para atender chamadas concorrentes de alta frequência.

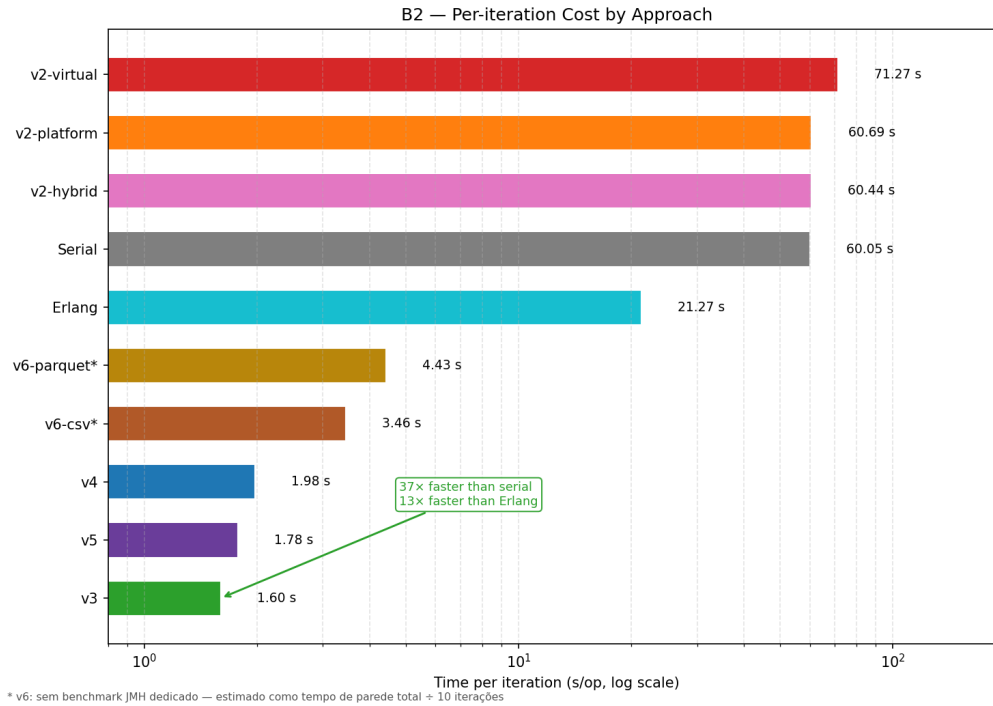


Figure 16: Custo por iteração (B2) em escala logarítmica, todas as abordagens. As variantes v2 e Serial ficam acima de 60 s/op, dominadas pelo parsing CSV. Erlang fica em 21,27 s/op ($\approx 13\times$ acima da v3) pelo custo aritmético da BEAM sem vetorização SIMD. v6-csv e v6-parquet (marcadas com *) não têm B2 dedicado — o valor mostrado é o tempo de parede total dividido por 10 iterações (ver Tabela 36 na Seção 8) — e ainda assim ficam acima de v3, v4 e v5, que caem para a faixa de 1,6–2,0 s/op ao eliminar a coordenação distribuída. v3 segue sendo a mais rápida, $37\times$ mais rápida que o Serial.

O `chart2_macro_throughput.png` (Seção 5) cobre apenas Serial, v2 e v3 e foi mantido como está. A Figura 17 é um gráfico novo e independente, comparando diretamente v3, v4 e v5 — as três únicas versões com macrobenchmark no mesmo coletor (ParallelGC), o que torna a comparação direta sem ressalvas de GC.

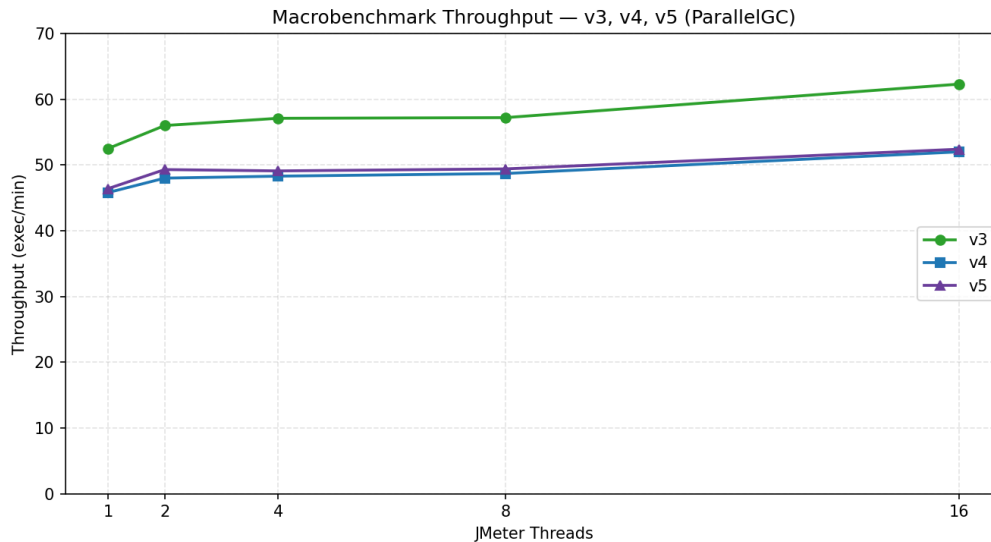


Figure 17: Throughput (execuções/min) por número de threads JMeter — v3, v4 e v5, todas com ParallelGC. v4 e v5 ficam muito próximas entre si e abaixo da v3 em todos os níveis de carga, consistente com o aumento de custo por iteração (B2) discutido nas Seções 6 e 7. v6 não aparece neste gráfico — não há macrobenchmark JMeter para o Spark (ver Seção 8).

9.2 Pontos Positivos e Negativos de Cada Abordagem

Serial

Positivos. Simples de implementar e depurar. Ausência de condições de corrida por definição. Baixo consumo de memória. Serve como baseline clara para medir ganhos das versões concorrentes.

Negativos. Subutiliza processadores multi-core. A thread principal acumula os dois papéis (I/O e computação) sem possibilidade de sobreposição.

v2 — Concorrência com Lotes e Futures

Positivos. Paraleliza a fase computacionalmente intensiva (atribuição de pontos a centróides) sem exigir sincronização explícita, graças à acumulação local por worker. A estrutura de `Future.get()` fornece garantia de happens-before clara. A variante hybrid demonstrou que virtual threads são adequadas para coordenadores que alternam entre I/O e espera.

Negativos. O parsing CSV sequencial na thread principal continua sendo o gargalo, tornando o ganho da paralelização invisível no tempo de parede. A criação de objetos `List.copyOf` e `PartialResult` por lote gera pressão de GC. A virtual thread em tarefas CPU-bound de curta duração introduz variância elevada no JMH (até ± 260 s), pois o `ForkJoinPool` não foi projetado para tarefas que nunca bloqueiam.

Erlang — Concorrência por Atores

Positivos. Race conditions são impossíveis por construção: processos não compartilham memória, eliminando a necessidade de locks, atomics ou `volatile`. O GC é por processo

e sem pausas globais: nenhum stop-the-world afeta todos os workers simultaneamente. Workers de longa duração mantêm os chunks em seus heaps locais sem overhead de cópia, graças à semântica de referência dos binários grandes da BEAM. O modelo de atores escala naturalmente para clusters distribuídos sem mudança de paradigma.

Negativos. A aritmética por ponto é $113\text{--}165\times$ mais lenta que em Java: a BEAM desempacota floats de binários elemento a elemento sem geração de código SIMD. Os schedulers operaram a 45,7% de utilização: a redução centralizada no processo principal (`collect_partial` + `recompute`) é um gargalo serial que mantém os workers ociosos $\approx 54\%$ do tempo. A reconstrução de listas de somas via `update_nth` gerou 2,1 milhões de coletas GC por alocação excessiva de células de lista — custo inexistente em Java, que acumula em arrays mutáveis locais. A ausência de servidor HTTP inviabilizou a comparação de throughput com JMeter nas mesmas condições das versões Java.

v3 — Dados em Memória com `CountDownLatch`

Positivos. Elimina o I/O das iterações, revelando o verdadeiro custo computacional do algoritmo. A divisão em N chunks fixos é mais simples que a criação dinâmica de lotes: sem `List.copyOf`, sem `Future` por lote. `CountDownLatch` é mais direto que coletar e iterar sobre uma lista de `Futures`. O `KMeansSampler` com `sharedPoints` estático permite que múltiplos threads JMeter compartilhem os dados sem alocar múltiplos GBs por thread.

Negativos. O dataset inteiro precisa caber na heap ($\approx 1,75$ GB para $\approx 5,75$ milhões de pontos com 38 features). Isso inviabiliza a abordagem para datasets maiores sem particionamento externo. O tempo de inicialização (leitura única do CSV) não é contabilizado nas métricas, o que pode ser enganoso se o processo for reiniciado frequentemente.

v4 — Concorrência Estruturada

Positivos. Corrige uma lacuna estrutural real da v3: exceções de worker agora propagam corretamente via `scope.join()`, com cancelamento automático de subtarefas irmãs. `ScopedValue` substitui a passagem de centróides por parâmetro com uma alternativa mais segura que `ThreadLocal` para virtual threads filhas. O confinamento local por tarefa é preservado sem alterações e segue validado por JCTest.

Negativos. Custo mensurável em relação à v3: B2 sobe $\approx 24\%$ ($1,60\text{ s} \rightarrow 1,98\text{ s}$) e o throughput cai em todos os níveis de carga testados. As APIs de *preview* exigem `-enable-preview` em toda a cadeia de build e execução (compilação, JMH, JCTest, JMeter).

v5 — Fork/Join

Positivos. Introduz *work-stealing* a nível de tarefa: divisão recursiva por limiar permite que threads ociosas roubem subtarefas de outras, vantagem estrutural em cargas desbalanceadas entre partições. B2 melhora levemente frente à v4 ($1,78\text{ s}$).

Negativos. Sem ganho aparente neste dataset uniforme — números de throughput quase idênticos aos da v4. A variância de B1 ($\pm 16,08\text{ s}$) é a maior desde a v2-virtual, mesmo efeito de ruído de agendamento em tarefas CPU-bound sobre `ForkJoinPool`. Nenhum teste de sensibilidade ao limiar de divisão foi realizado.

v6 — Spark (Processamento Distribuído)

Positivos. Modelo de programação radicalmente diferente das versões anteriores (transformações distribuídas sobre um `Dataset` particionado em vez de threads coordenadas sobre memória compartilhada), relevante para cargas que não cabem em uma única máquina. `mapToPair+reduceByKey` expressa o mesmo padrão de acumulação local das versões anteriores em um paradigma declarativo.

Negativos. Mais lento que v3–v5 neste dataset, que cabe confortavelmente em memória numa única máquina — o overhead de coordenação distribuída não se paga nesta escala. JMH, JMeter e JCSstress não se aplicam, reduzindo a comparabilidade direta com as versões anteriores. Sob pressão de memória, o cache degrada silenciosamente para recomputação em vez de falhar de forma explícita (ao contrário da v3, que ou cabe no heap ou lança `OutOfMemoryError`). Contraintuitivamente, o formato Parquet foi $\approx 28\%$ mais lento que o CSV nesta carga.

9.3 Lições Aprendidas

Medir antes de paralelizar. O JFR mostrou que $\approx 82\%$ do tempo serial era parsing CSV. Paralelizar apenas o loop aritmético ($\approx 8\%$ do total) dificilmente produziria ganho perceptível sem antes eliminar o gargalo de I/O.

Gargalos migram. Com o I/O eliminado na v3, o gargalo passou a ser `squaredDistance` e `closestCentroid`. Em iterações futuras, esse é o ponto a otimizar (SIMD, vetorização, redução de dimensionalidade).

Virtual threads não são bala de prata para CPU-bound. Virtual threads brilham com bloqueio I/O, liberando o carrier para reuso. Em tarefas puramente aritméticas, não oferecem vantagem sobre platform threads e podem introduzir variância de escalonamento indesejada.

Acumulação local elimina sincronização. O padrão map-reduce local — cada worker acumula em arrays próprios, a thread coordenadora reduz no final — evitou locks, atomics e `volatile`. O JCSstress confirmou a ausência de race conditions nas operações reais e sua presença ao remover a proteção (JCS3, JCS4).

O modelo de atores não é universal. K-Means é CPU-bound com comunicação mínima entre iterações — perfil que favorece memória compartilhada. O modelo de atores Erlang brilha em sistemas distribuídos, I/O intensivo e alta disponibilidade, onde o isolamento de processos é vantagem estrutural, não custo.

Robustez estrutural tem custo mensurável. A v4 trocou uma lacuna real de tratamento de erro (exceção de worker silenciosamente mascarada na v3) por um aumento de $\approx 24\%$ no custo por iteração — APIs mais seguras (`StructuredTaskScope`, `ScopedValue`) não são gratuitas enquanto ainda são *preview*.

Work-stealing não ajuda cargas uniformes. A v5 não superou a v4 neste dataset sintético e bem distribuído entre partições — *work-stealing* só compensa seu overhead de agendamento quando há desbalanceamento real de carga, o que não é o caso aqui.

Mudar o modelo de execução muda o cálculo de custo-benefício. O Spark (v6) foi mais lento que todas as versões Java de memória compartilhada nesta escala — o overhead de coordenação distribuída (serialização, broadcast, gerenciamento de partições) só se paga quando o dataset não cabe em uma única máquina.

Formato de armazenamento também é uma variável de desempenho. Contra a intuição de que Parquet (colunar, comprimido) seria mais rápido que CSV, ele foi $\approx 28\%$

mais lento nesta carga — o custo de decodificação de dicionário/delta por linha superou a economia de I/O bruto. Mais uma confirmação da lição "medir antes de assumir".

Sistemas distribuídos falham de formas que sistemas single-JVM não falham.

Sob pressão de memória, o cache do Spark degrada silenciosamente para recomputação em vez de lançar um erro explícito — ao contrário da v3, onde o dataset ou cabe no heap ou a JVM falha de forma visível.

10 Repositórios

Os repositórios do projeto estão disponíveis no GitHub. Cada um corresponde a um componente do trabalho e possui tags marcando as versões entregues.

Implementação Java

Contém o algoritmo K-Means serial e as variantes concorrentes (v2-platform, v2-virtual, v2-hybrid, v3), além dos benchmarks JMH, testes JCStress, macrobenchmark JMeter e profile JFR.

- **URL:** <https://github.com/rarycoringa/concurrent-kmeans-java-application>
- **Tags:** v1 (serial), v2-platform, v2-virtual, v2-hybrid, v3 (dados em memória)

Implementação Erlang

Contém a implementação concorrente em Erlang com o modelo de atores da BEAM, benchmarks com `timer:tc` e profile com `erlang:statistics`.

- **URL:** <https://github.com/rarycoringa/concurrent-kmeans-erlang-application>
- **Tags:** v1 (concorrente, dados em memória)

Gerador de Dataset

Scripts Python para geração do dataset sintético de pedidos utilizado em todos os experimentos ($\approx 5,75$ milhões de linhas, 38 features).

- **URL:** <https://github.com/rarycoringa/concurrent-kmeans-dataset-generator-python-s>
- **Tags:** nenhuma